Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Гантөмөрийн Алтай

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Бакалаврын төгсөлтийн ажил

Улаанбаатар хот

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Компьютерын ухааны салбар

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Мэргэжлийн индекс: 071405000000002306
Мэргэжил: Программ хангамжийн инженерчлэл

Удирдагч: Магистр (M.Sc) Э.Батцэцэг
Зөвлөгч: Доктор (Ph.D), Дэд профессор Г.Ганбат
Гүйцэтгэгч: Г.Алтай

Улаанбаатар хот
2026 он 6 сар

Батлав. Компьютерын ухааны салбарын эрхлэгч:

..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Удирдагч:

..... /Магистр (M.Sc) Э.Батцэцэг/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт"

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал хэсэг	5%	2026-2-4
2	Судалгааны хэсэг	15%	2026-2-21
3	Шинжилгээний хэсэг	15%	2026-3-4
4	Зохиомжийн хэсэг	20%	2026-4-1
5	Хөгжүүлэлтийн хэсэг	40%	2026-4-29
6	Дүгнэлт	5%	2026-5-27

Төлөвлөгөөг боловсруулсан оюутан: ..... /Г.Алтай/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Удиртгал	2026-2-4	
2	Судалгааны хэсэг	2026-2-21	
3	Шинжилгээний хэсэг	2026-3-4	
4	Зохиомжийн хэсэг	2026-4-1	
5	Хөгжүүлэлтийн хэсэг	2026-4-29	
6	Дүгнэлт	2026-5-27	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Магистр (M.Sc) Э.Батцэцэг/

ЗӨВШӨӨРӨЛ

Оюутан Г.Алтай-н бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

ШҮҮМЖИЙН ХУУДАС

Компьютерын ухааны салбарын төгсөх курсийн оюутан Г.Алтайгын "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт төгсөлтийн ажлын шүүмж:

1. Төслөөр дэвшүүлсэн асуудал, үүнтэй холбоотой онолын материал уншиж судалсан байдал; Энэ талаар хүмүүсийн хийсэн судалгаа, түүний үр дүнг уншиж тусгасан эсэх:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

2. Төслийн ерөнхий агуулга, шийдсэн зүйлс, хүрсэн үр дүн; Өөрийн санааг гарган, харьцуулалт хийн, дүгнэж байгаа чадвар:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

3. Эмх цэгцтэй, стандарт хангасан өөрөөр хэлбэл диплом бичих шаардлагуудыг биелүүлсэн эсэх; Төсөлд анзаарагдсан алдаанууд, зөв бичгийн болон өгүүлбэр зүйн гэх мэт /Хуудас дугаарлагдаагүй, зураг хүснэгтийн дугаар, тайлбар байхгүй, шрифт хольсон, хувилсан зүйл ихээр оруулсан/:

.....

.....

.....

.....

.....

.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлс; Цаашид анхаарах хэрэгтэй зүйлс:

.....

.....

.....

.....

.....

.....

5. Төслийн талаар онцолж тэмдэглэх зүйлс:

.....

.....

.....

.....

.....

.....

6. Ерөнхий оноо (5 оноо)

.....

Шүүмж бичсэн: ..... /Магистр (M.Sc) О.Нэр/

Ажлын газар: .....

Хаяг (Утас) .....

Зохиогч эрхийн хамгаалал

Миний бие Г.Алтай, "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ гол эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

"*Thanks to my solid academic training, today I can write hundreds of words on virtually any topic without possessing a shred of information, which is how I got a good job in journalism.*"

Dave Barry

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Хураангуй

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Г.Алтай
bbyyydriver@gmail.com

Түлхүүр үгс: Үүлэн хадгалалтын систем, веб систем, IOS native мобайл app

Талархал

Энэхүү дипломын ажлыг хийж дуусгахад хамгийн гол нөлөө үзүүлсэн, сэдэв бүрийг нягталж, үнэтэй зөвөлгөө өгч, алдааг маань засаж, удирдан чиглүүлж байсан удирдагч багш Э.Батцэцэг багшдаа хамгийн түрүүнд чин сэтгэлээсээ баярлалаа гэж хэлмээр байна. Мөн дипломын ажлын зөвлөх багш болох Г.Ганбат багшдаа үнэт цагаа зарцуулж, зааж, сургаж, орхигдуулсан зүйлсийг сануулж, хэрэгтэй зөвлөгөөнүүд өгч байсанд гүн талархлаа илэрхийлье.

Намайг бакалаврын дипломын ажил бичиж чадах хэмжээнд хүртэл зааж сургасан МХТС -н Компьютерын Ухааны салбарын багш нар, “Программ хангамж” мэргэжлийн хөтөлбөрийн хүрээнд хичээл зааж байсан ШУТИС -н багш нартаа баярлалаа. Суралцахад аюулгүй, цэвэрхэн орчныг бүрдүүлж өгч байсан МХТС -н ажилчдад баярлалаа.

Академик мэдлэгээ практик дээр туршин, дипломын ажлын хүрээнд шинэ технологиудыг судлан, мэргэжлийн хувьд туршлагажих боломж олгож байсан Грэйпсити Монгол компанийн “Технелогийн Газар” -н захирал болон ахлах инженертээ талархлаа илэрхийлмээр байна.

Эцэст нь, би дипломын ажилдаа их цаг зарцуулах боломжийг олгож, олон талаар чухал дэмжлэг үзүүлж байсан гэр бүлийнхэн болон урам зориг, эрч хүчээр урамшуулж байсан найзууддаа ч бас чин сэтгэлийн талархлаа илэрхийлэх ёстой.

Товчилсон үгс

Гарчиг

Зохиогч эрхийн хамгаалал **i**

Хураангуй **iii**

Талархал **iv**

Товчилсон үгс **v**

Удиртгал **1**

1 Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа **4**

1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт 4

1.2 Ижил төст системүүдийн судалгаа 6

1.2.1 Судалгааны үнэлгээний шалгуур 6

1.2.2 Google Drive: Нийтийн Cloud Storage-ийн Төлөөлөл 6

1.2.3 Nextcloud: Self-Hosted Personal Cloud-ийн Төлөөлөл 9

1.2.4 ownCloud Infinite Scale (oCIS): Микросервис Чиглэлийн Personal Cloud 12

1.2.5 Нэгдсэн Харьцуулалт 16

1.3 Ашиглах технологиуд 17

1.4 Бүлгийн дүгнэлт 18

2 Хувийн үүлэн хадгалалтын системийн шинжилгээ **19**

2.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй 19

2.2 Функциональ шаардлага 19

2.2.1 Usecase диаграмм 20

2.2.2 Юзкейсийн тодорхойлолт 22

Хэрэглэгч үүсгэх (UC-001) 22

Хэрэглэгч устгах (UC-002) 23

Эрх олгох (UC-003) 24

Системд нэвтрэх (UC-004) 25

Файл болон хавтас харах (UC-005) 26

Файл байршуулах (UC-006) 27

Файл татаж авах (UC-007) 28

Хавтас үүсгэх (UC-008) 29

Файл/хавтас устгах (UC-009) 30

Файл/хавтас шилжүүлэх (UC-010) 31

2.3 Функциональ бус шаардлага 31
2.4 Ажлын хүрээ 32
2.4.1 Scope (Хамрах хүрээ) 32
2.4.2 Out of Scope (Хийхгүй зүйлс) 32
2.5 Бүлгийн дүгнэлт 33

3 Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт 34
3.1 Системийн архитектур 34
3.2 Зохиомжийн үндсэн компонентууд 34
3.3 Өгөгдлийн ерөнхий схем 35
3.4 Системийн прототип 36
3.5 Хөгжүүлсэн системийн интерфейс 39
3.6 Системд хийгдсэн тест 39
3.7 Системийн нэвтрүүлэлт 39
3.8 Бүлгийн дүгнэлт 39

Ерөнхий дүгнэлт 41

A Серверийн API-ийн жишээ 42
A.1 REST endpoint-ууд 42

Ашигласан материалын жагсаалт 43

Зургийн жагсаалт

1.1 Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт 5
1.2 Colossus control plane-ийн ерөнхий бүтэц (эх сурвалж: Google Cloud Blog) [2] 8
1.3 Nextcloud-ийн хандалтын ба хадгалалтын давхаргын ерөнхий зураглал (эх сурвалж: Nextcloud Architecture Whitepaper) [13] 10
1.4 Docker орчинд ажиллуулсан Nextcloud-ийн веб интерфэйсийн дэлгэцийн зураг 11
1.5 Docker дээр ажиллуулсан Nextcloud AIO контейнерүүдийн санах ойн хэрэглээний дэлгэцийн зураг 11
1.6 ownCloud Infinite Scale-ийн system context зураг (эх сурвалж: ownCloud docs) [23] 13

2.1 Usecase диаграм 21

3.1 Системийн архитектур 34
3.2 Хувийн үүлэн хадгалалтын системийн ERD 36
3.3 Анхны тохиргооны дэлгэц 37
3.4 Нэвтрэх дэлгэц 37
3.5 Энгийн хэрэглэгчийн үндсэн хуудас 38
3.6 Админ хэрэглэгчийн нүүр хуудас 38

Хүснэгтийн жагсаалт

1.1 Google Drive, Nextcloud, ownCloud oCIS системүүдийн нэгдсэн үнэлгээ . 16

2.1 UC-001 usecase-ийн тодорхойлолт 22
2.2 UC-002 usecase-ийн тодорхойлолт 23
2.3 UC-003 usecase-ийн тодорхойлолт 24
2.4 UC-004 usecase-ийн тодорхойлолт 25
2.5 UC-005 usecase-ийн тодорхойлолт 26
2.6 UC-006 usecase-ийн тодорхойлолт 27
2.7 UC-007 usecase-ийн тодорхойлолт 28
2.8 UC-008 usecase-ийн тодорхойлолт 29
2.9 UC-009 usecase-ийн тодорхойлолт 30
2.10 UC-010 usecase-ийн тодорхойлолт 31

Удиртгал

Өнөөгийн дижитал эрин үед үүлэн хадгалалт (Cloud Storage) нь хэрэглэгчдийн өдөр тутмын амьдралын салшгүй хэсэг болжээ. Хэрэглэгчид зураг, бичлэг, ажлын болон хувийн чухал файлуудаа интернетийн тусламжтайгаар хаанаас ч хандаж ашиглах шаардлагатай байдаг. Энэхүү хэрэгцээг хангахын тулд Google Drive, iCloud, OneDrive зэрэг төвлөрсөн үүлэн хадгалалтын үйлчилгээ өргөн хэрэглэгддэг болсон.

Гэвч эдгээр үйлчилгээний хувьд хэрэглэгчийн өгөгдөл тухайн үйлчилгээ үзүүлэгч компанийн сервер дээр хадгалагддаг бөгөөд хэрэглэгч өөрийн мэдээллийн бүрэн эзэмшил, хяналтыг алдах эрсдэлтэй байдаг. Мөн хадгалах багтаамж нэмэгдэх тусам тогтмол төлбөр төлөх шаардлага гардаг нь урт хугацаанд хэрэглэгчдэд эдийн засгийн дарамт учруулж болзошгүй.

Иймд хэрэглэгч өөрийн эзэмшлийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл өөрийн сервер дээр суурилсан хувийн үүлэн хадгалалтын системийг хөгжүүлэх нь чухал асуудал болж байна. Энэхүү ажлын хүрээнд хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах, веб болон мобайл клиентээр дамжуулан удирдах боломжтой, хөнгөн бүтэцтэй (lightweight) хувийн үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байна.

Зорилго

Энэхүү төслийн зорилго нь хэрэглэгч өөрийн эзэмшлийн техник хангамжид суурилсан, өөрөө байршуулж (self-hosted) ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийг зохион бүтээж хэрэгжүүлэхэд оршино. Тус систем нь хэрэглэгчдэд веб болон мобайл клиент ашиглан өөрийн сервер дээр хадгалагдсан файлуудыг удирдах, хандах боломжийг олгоно.

Төслийн зорилтууд

Дэвшүүлсэн зорилгод хүрэхийн тулд дараах зорилтуудыг хэрэгжүүлнэ. Үүнд:

1. Хувийн үүлэн хадгалалтын системүүдийн архитектур болон технологийн харьцуулсан судалгаа хийх;
2. Нөөц багатай төхөөрөмж (Raspberry Pi зэрэг) дээр ажиллах боломжтой хөнгөн серверийн программ хангамжийг зохион бүтээх;
3. Файл хадгалах, татах, зохион байгуулах үндсэн үйлдлүүдийг дэмжих серверийн REST API хөгжүүлэх;

4. Хэрэглэгч веб хөтөч ашиглан файлуудаа удирдах боломжтой веб клиент аппликейшн хөгжүүлэх;
5. iOS үйлдлийн системд зориулсан хэрэглэгчийн мобайл аппликейшн боловсруулах;
6. Хэрэглэгч өөрийн серверийг интернэт орчинд байршуулан алсаас хандах боломжийг бүрдүүлэх архитектурын шийдлийг боловсруулах.

Судлах үндэслэл

Сүүлийн жилүүдэд үүлэн хадгалалтын хэрэглээ огцом өсөж, 2025 оны байдлаар дэлхий даяар үүлэн хадгалалтын хэрэглэгчдийн тоо 2.3 тэрбумыг [1] давсан байна. Гэвч төвлөрсөн үүлэн үйлчилгээний хэрэглээ өсөхийн хэрээр дараах асуудлууд тулгарч байна.

1. **Өгөгдлийн эзэмшил ба нууцлал**: Хэрэглэгчийн мэдээлэл гуравдагч этгээдийн сервер дээр хадгалагддаг тул өгөгдлийн бүрэн хяналт хэрэглэгчид байхгүй.
2. **Тогтмол үйлчилгээний зардал**: Үүлэн хадгалалтын үйлчилгээний багтаамж нэмэгдэх тусам хэрэглэгч тогтмол төлбөр төлөх шаардлагатай болдог.

Иймд хэрэглэгч өөрийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл self-hosted хувийн үүлэн системийг хөгжүүлэх нь технологийн болон практик ач холбогдолтой юм.

Судлагдсан байдал

Хувийн үүлэн хадгалалтын системийн чиглэлээр Nextcloud, Seafile, Syncthing зэрэг нээлттэй эхийн төслүүд өргөн хэрэглэгддэг. Эдгээр системүүд нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах боломжийг олгодог боловч ихэвчлэн том хэмжээний системийн архитектуртай, олон төрлийн нэмэлт функцүүдийг агуулсан байдаг.

Тухайлбал, Nextcloud нь хамтын ажиллагаа, календарь, мэйл зэрэг олон төрлийн үйлчилгээг нэг дор багтаасан цогц платформ бол Seafile нь файл синхрончлолын өндөр гүйцэтгэлд төвлөрсөн систем юм. Гэвч эдгээр системүүдийг бага нөөцтэй төхөөрөмж дээр ажиллуулахад тохиргоо хийхэд төвөгтэй, системийн шаардлага өндөр байх зэрэг сул талууд ажиглагддаг.

Иймд энэхүү төслөөр илүү хөнгөн бүтэцтэй, хувийн хэрэглээнд зориулсан үүлэн хадгалалтын системийг зохион бүтээхийг зорьсон.

Шинэлэг тал

Хувийн үүлэн хадгалалтын системийн шинэлэг тал:

1. **Хөнгөн архитектур**: Нөөц багатай төхөөрөмжүүд дээр ажиллах боломжтой энгийн бөгөөд үр ашигтай серверийн архитектур ашиглана.
2. **Self-hosted шийдэл**: Хэрэглэгч өөрийн сервер дээр системийг байршуулж өгөгдлөө бүрэн хянах боломжтой.
3. **Олон төрлийн клиент**: Веб болон мобайл клиент ашиглан файлуудад хандах боломжтой.

Технологийн ач холбогдол

Энэхүү систем нь технологийн хувьд дараах ач холбогдолтой:

1. **Өгөгдлийн бүрэн эзэмшил**: Хэрэглэгч өөрийн сервер дээр өгөгдлөө хадгалснаар мэдээллийн бүрэн хяналтыг хадгална.
2. **Хөнгөн серверийн шийдэл**: Орчин үеийн асинхрон серверийн архитектур ашиглан өндөр гүйцэтгэлтэй систем бий болгоно.
3. **Олон платформын клиент**: Веб болон мобайл клиентүүдээр дамжуулан хэрэглэгчийн хүртээмжийг нэмэгдүүлнэ.

Хамрах хүрээ

Тус төсөл нь дараах хүрээг хамарна:

- Өөрийн өгөгдлийн нууцлалыг эрхэмлэдэг хувь хүн болон гэр бүлийн хэрэглэгчид;
- Өндөр өртөг бүхий үүлэн үйлчилгээнээс татгалзаж, өөрийн нөөцийг ашиглах хүсэлтэй технологи сонирхогчид;

БҮЛЭГ 1

Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа

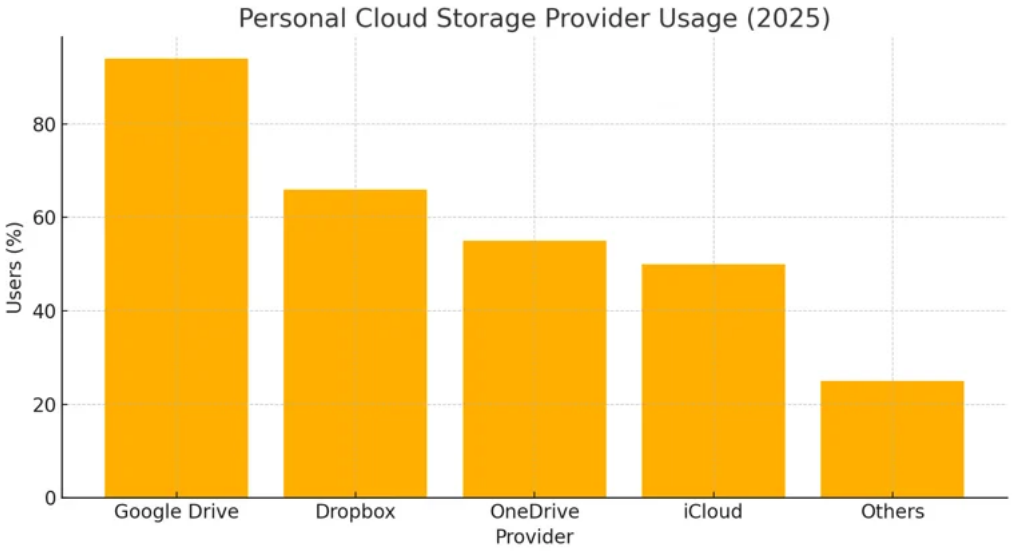
1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт

Үүлэн хадгалалт (*Cloud Storage*) нь өгөгдлийг локал төхөөрөмж дээр бус, сүлжээгээр холбогдсон алсын сервер дээр хадгалах технологи юм. Хэрэглэгч интернэтээр дамжуулан өөрийн файлуудад хаанаас ч, ямар ч төхөөрөмжөөс хандах боломжтой байдаг. Энэхүү технологи нь өгөгдлийг төвлөрсөн байдлаар хадгалах, нөөцлөх, хуваалцах зэрэг давуу талуудыг олгодог.

Үүлэн хадгалалтын системийг ерөнхийд нь дараах гурван төрөлд ангилдаг:

- **Нийтийн (Public Cloud)** – Гуравдагч байгууллагын эзэмшлийн сервер дээр өгөгдлийг хадгалдаг (жишээ нь: Google Drive, Dropbox). Хэрэглэхэд хялбар боловч өгөгдлийн хяналт харьцангуй бага байдаг.
- **Хувийн (Private Cloud)** – Хэрэглэгч өөрийн сервер дээр системийг ажиллуулж, өгөгдлөө бүрэн хянах боломжтой. Аюулгүй байдал болон нууцлал өндөр боловч тохиргоо, удирдлага шаарддаг.
- **Холимог (Hybrid Cloud)** – Нийтийн болон хувийн үүлэн системийг хослуулсан хэлбэр. Уян хатан байдал сайтай боловч архитектурын хувьд илүү төвөгтэй.

Сүүлийн жилүүдэд өгөгдлийн нууцлал, аюулгүй байдлын асуудал нэмэгдэж байгаатай холбоотойгоор хувийн үүлэн хадгалалтын системд анхаарах хандлага нэмэгдэж байна. Хэрэглэгч өөрийн өгөгдлийг бүрэн хянах, гуравдагч этгээдээс хамааралгүй байх шаардлага нь ийм төрлийн системийг хөгжүүлэх үндэслэл болж байна. 2025 оны судалгаагаар [1] дэлхий дээр нийт 2.3 тэрбум хүн үүлэн хадгалалтын үйлчилгээг гуравдагч системээс авч байгаа гэсэн судалгаа гарсан. Үүний хэрэглэгчдийн ашиглалтын үзүүлэлт болон заах зээлийн хуваалт нь Google Drive 94%, Dropbox 66%, OneDrive 55%, iCloud 50% (хэрэглэгчид эдгээр платформуудыг зэрэг хэрэглэсэн байна). Дараах зурагт хэрэглээний түвшинг графикаар харуулав.



Зураг 1.1: Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт

Судалгааны үр дүнгээс үзэхэд хэрэглэгчдийн 55% дор хаяж 3 үүлэн хадгалалтын үйлчилгээг зэрэг хэрэглэж байна. Хүмүүс ихэвчлэн файлуудаа платформуудад хуваан, тухайн үйлчилгээний онцлогт тааруулан ашигладаг. Google Photos-ийг зураг хадгалахдаа ашиглаж, баримт бичгийг Dropbox, iPhone утасныхаа өгөгдлийг iCloud дээр хадгалах нь түгээмэл ажиглагдаж байна. Хэрэглэгчдийн дунд өгөгдлийн аюулгүй байдлын халдлага болон асуудал энэхүү салбарт хамгийн том асуудлуудын нэгт орж байгаа бөгөөд үүнд зориулж үйлчилгээ үзүүлэгч байгууллагууд илүү найдвартай шифрлэлт, хоёр шатлалт баталгаажуулалт гэх мэт хамгаалалтын шийдлүүдийг санал болгож байна. 2024 оны Англи улсад хэрэглэгчдээс авсан судалгаагаар үүлэн хадгалалтын хэрэглэгчдийн 43% нь тухайн үүлэн хадгалалтын үйлчилгээ үзүүлэгч байгууллага нь өөрсдийнх нь хувийн өгөгдлийг шинжилдэг гэдэгт санаа зовнидог гэсэн хариулт өгсөн байна. Үүлэн хадгалалтын үйлчилгээнүүд нь ойрын ирээдүйд хиймэл оюунтай ашиглан файл системийн автомат цэгцлэлт, ухаалаг хайлт, өгөгдөл шошгожуулах, олон платформ дээр дэмжигдэх байдал, сүлжээгүй үед хэрхэн sync хийгдэх гэсэн зүйлс дээр нэмэлт сайжруулалт хийгдэн өрсөлдөөн үүсэх төлөвтэй байгаа ажээ. **Энэхүү дипломын ажлаар миний бие хувийн гэсэн ангилалд багтах үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байгаа юм.**

1.2 Ижил төст системүүдийн судалгаа

Энэ хэсгийн зорилго нь personal cloud storage системийн зохиомжид бодитой нөлөө үзүүлэх ижил төст платформуудыг харьцуулж, тэдгээрийн архитектур болон шийдлийн логикоос хэрэгжүүлэхүйц дүгнэлт гаргах явдал юм. Судалгаанд cloud storage ба file syncing гэсэн хоёр чиглэлийг ерөнхийд нь ялган авч үзсэн боловч дипломын ажлын үндсэн зорилт нь **personal cloud storage system** хөгжүүлэх тул гүн шинжилгээг cloud storage ангиллын төлөөлөгчдөд төвлөрүүлэв.

Тодруулбал Google Drive (public cloud), Nextcloud болон ownCloud Infinite Scale (self-hosted cloud) системүүдийг сонгон, дараах асуудлууд дээр төвлөрч судаллаа: архитектурын суурь давхарга, өгөгдлийн хяналт ба бодлогын удирдлага, синхрончлолын ажиллагааны шийдэл, аюулгүй байдал ба ажиллагааны төвөгшил. Харин Apple iCloud, Microsoft OneDrive, Seafile, Syncthing/Resilio Sync зэрэг платформуудыг зөвхөн судалгааны хүрээ, контекстийг тодорхойлох түвшинд дурдав.

1.2.1 Судалгааны үнэлгээний шалгуур

Сонгосон cloud storage системүүдийг дараах шалгуураар үнэлэв.

1. **Системийн зорилго**: тухайн платформ ямар үндсэн асуудлыг шийдэхэд төвлөрдөг вэ.
2. **Архитектурын бүтэц**: өгөгдөл хадгалах болон удирдах үндсэн давхаргууд хэрхэн зохион байгуулагдсан бэ.
3. **Өгөгдлийн хяналт**: өгөгдөл болон хандалтын бодлогыг хэрэглэгч хэр хэмжээнд өөрөө удирдах вэ.
4. **Аюулгүй байдал ба сэргээх боломж**: шифрлэлт, хувилбарын түүх, алдагдсан өгөгдөл сэргээх чадвар.
5. **Ашиглалт ба өргөтгөл**: өдөр тутмын хэрэглэхэд хялбар байдал болон системийг цааш өргөтгөх боломж.

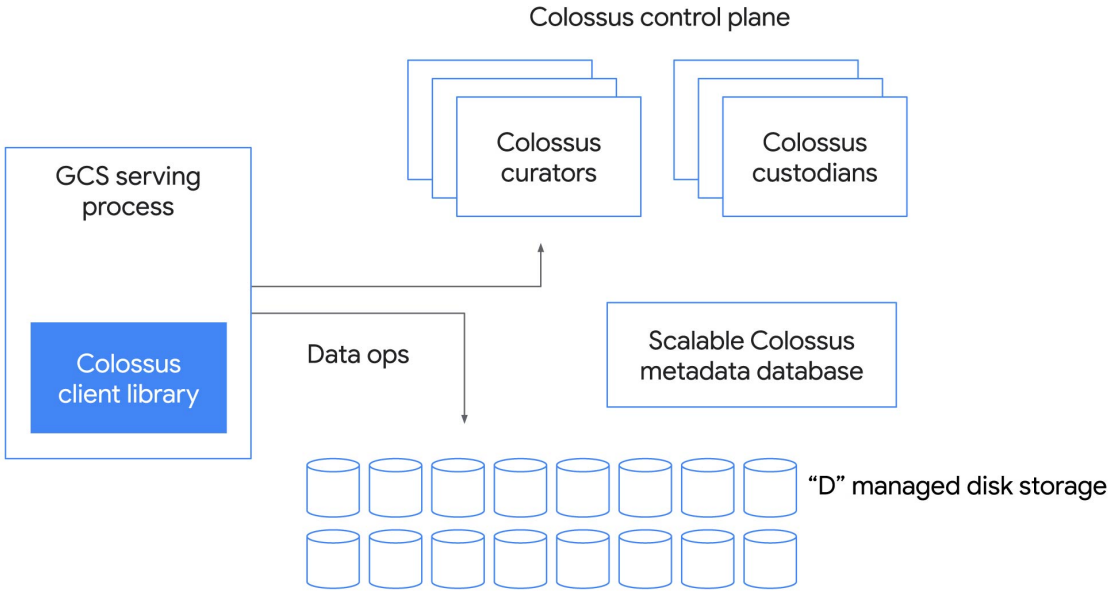
1.2.2 Google Drive: Нийтийн Cloud Storage-ийн Төлөөлөл

Google Drive нь personal cloud storage системтэй шууд ижил ангилалд хамаарахгүй ч, том хэмжээний cloud storage үйлчилгээг хэрхэн найдвартай ажиллуулж болдгийг харуулах сайн жишээ юм. Энэ платформын үнэ цэнэ нь зөвхөн файл хадгалахдаа биш, харин Google Workspace-ийн баримт бичиг, хамтын ажиллагаа, хуваалцах эрх, хайлт, хувилбарын түүх зэрэг олон урсгалыг нэг экосистемд уялдуулан тогтвортой

ажиллуулдгаараа илэрдэг. Тиймээс энэхүү судалгаанд Google Drive-ийг “боловсорсон public cloud storage систем ямар шинжтэй байх ёстой вэ” гэдэг өнцгөөс авч үзэв.

Google-ийн тайлбарласнаар Drive зэрэг үйлчилгээний ард Colossus, Spanner, Borg гэсэн нийтлэг суурь давхаргууд ажилладаг [2]. Энд Colossus нь файлын үндсэн хадгалалтыг, Spanner нь глобал хэмжээнд нийцтэй мета өгөгдлийн удирдлагыг, Borg нь тооцооллын нөөцийн зохион байгуулалтыг дэмждэг. Энэ нь хэрэглэгчийн хувьд нэг энгийн файл хадгалах үйлчилгээ мэт харагдавч, цаанаа олон дата төв, олон машин, тасралтгүй ачаалал, алдааны үед ч ажиллагаагаа үргэлжлүүлэхээр зохион байгуулагдсан систем гэсэн үг юм. Google Workspace-ийн аюулгүй байдлын баримт бичигт өгөгдлийг дамжуулах үед болон хадгалж байх үед шифрлэдэг, мөн өгөгдлийг олон диск, олон сервер, олон дата төв дээр ойролцоо хугацаанд давхар хадгалж өгөгдөл алдах эрсдэлийг бууруулдаг гэж тайлбарласан байдаг [3].

Өгөгдлийн хяналтын хувьд Google Drive нь self-hosted систем шиг бүрэн өмчлөл өгөхгүй ч байгууллагын бодлогын түвшний хэд хэдэн чухал тохиргоог санал болгодог. Жишээлбэл, Google Workspace админ нь тодорхой төрлийн өгөгдлийг АНУ, Европын Холбоо, эсвэл бүсийн онцгой шаардлагагүй гэсэн сонголтоор байршуулах боломжтой [4]. Мөн Client-side Encryption (CSE)-ийг ашигласнаар байгууллага өөрийн шифрлэлтийн түлхүүрийг удирдаж, өгөгдлийг Google-ийн үүлэнд хүрэхээс өмнө шифрлэх боломжтой. Ингэснээр Google серверүүд болон гуравдагч талууд тухайн түлхүүргүйгээр өгөгдлийг тайлж унших боломжгүй болдог [5]. Энэ нь Google Drive-ийн хяналтын загвар нь “платформын өмчлөл Google-д, харин бодлогын нарийвчилсан удирдлага тодорхой түвшинд хэрэглэгч байгууллагад” гэсэн хэлбэртэйг харуулж байна.



Зураг 1.2: Colossus control plane-ийн ерөнхий бүтэц (эх сурвалж: Google Cloud Blog) [2]

Зураг 1.2-т Colossus-ийн control plane болон өгөгдлийн урсгалыг энгийн байдлаар харуулжээ. Зүүн талын client library нь үйлчилгээний хүсэлтийг Colossus руу дамжуулдаг. Дээд талд байрлах curator болон custodian төрлийн бүрэлдэхүүнүүд нь мета өгөгдөл, хадгалалтын зохион байгуулалт, тэнцвэржүүлэлт, нөхөн сэргээх зэрэг удирдлагын ажлыг хариуцна. Харин доод талын managed disk storage нь бодит өгөгдлийг хадгалж байгаа хэсэг юм. Өөрөөр хэлбэл зураг нь удирдлагын үйлдэл ба өгөгдөл дамжуулах үйлдлийг салгаснаар системийг их хэмжээнд өргөтгөж, нэг хэсэгт асуудал гарсан ч бүхэл үйлчилгээ саатах эрсдэлийг багасгаж болохыг харуулж байна [2]. Энэ нь personal cloud системийн хувьд ч хэрэгтэй санаа бөгөөд мета өгөгдөл, синк логик, файлын бодит хадгалалтыг нэг дор хэт нягт холбохгүй байх нь зөв шийдэл гэдгийг сануулж байна.

Google Drive-ийн хэрэглэгчийн түвшний хүчтэй тал нь синхрончлол болон сэргээх урсгал дээр илүү тод харагддаг. Drive API нь өөрчлөлтийн жагсаалтыг start page token-оос эхлэн дараалаар авах, өөрчлөлт гарсныг 'changes.watch'-аар мэдэгдэх, том хэмжээний файлыг тасалдсан тохиолдолд resumable upload-аар үргэлжлүүлэх, мөн файлын revision-уудыг удирдах боломж олгодог [6, 7, 8, 9, 10]. Иймээс Google Drive-ээс авах хамгийн чухал сургамж нь "файл хадгалах" боломж дангаараа хангалтгүй, харин өөрчлөлт мөрдөх, тасалдсан дамжуулалтыг сэргээх, хувилбарын түүх хадгалах зэрэг урсгалууд системийн үндсэн хэсэг байх ёстой гэсэн санаа юм.

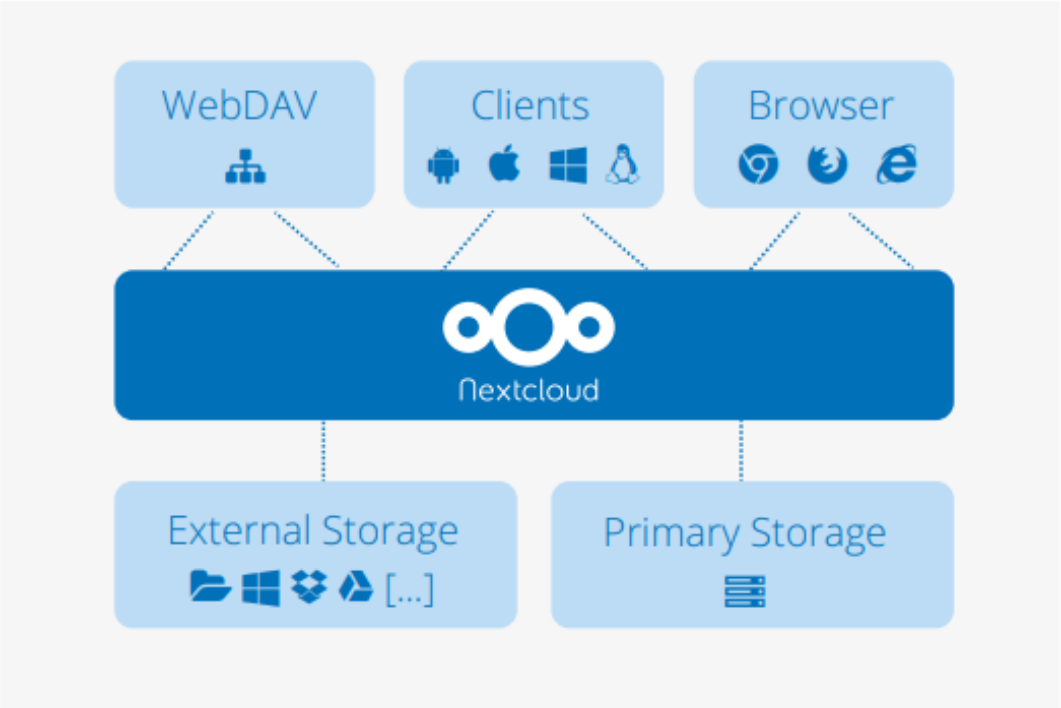
Дүгнэж хэлбэл Google Drive нь өгөгдлийн бүрэн өмчлөлөөс илүүтэйгээр production түвшний найдвартай ажиллагаа, синк урсгалын боловсронгуй байдал, аюулгүй байдлын бодлогын зохион байгуулалтаараа энэ судалгаанд үнэ цэнтэй жишиг болж байна. Харин энэхүү дипломын ажлын зорилго нь ийм төрлийн ажиллагааны давуу талыг авч, self-hosted орчинд хэрэглэгчийн өгөгдлийн хяналтыг илүү өндөр түвшинд хадгалах шийдэл боловсруулахад оршино.

1.2.3 Nextcloud: Self-Hosted Personal Cloud-ийн Төлөөлөл

Nextcloud нь self-hosted personal cloud-ийн хамгийн төлөөлөхүйц шийдлүүдийн нэг юм. Үүний гол ялгарах тал нь public cloud үйлчилгээ шиг файл хадгалах, хуваалцах, олон төхөөрөмжөөс хандах боломжийг өгдөг атлаа системийн эзэмшил, байршуулалт, бодлогын хяналтыг хэрэглэгч эсвэл байгууллагын талд үлдээдэгт оршино. Nextcloud Files-ийн албан ёсны танилцуулгад веб интерфэйс, desktop client, mobile app-аар хаанаас ч хандах боломжтой бөгөөд гуравдагч тал руу өгөгдөл алдахгүйгээр full control хадгална гэж тодорхойлсон байдаг [11]. Иймээс Nextcloud-ийг энэ судалгаанд "public cloud-ийн хэрэглээний мэдрэмжийг self-hosted орчинд ойртуулсан шийдэл" гэж үзэж болно.

Түүхэн хувьд Nextcloud нь ownCloud-той шууд холбоотой. Nextcloud-ийн албан ёсны 2016 оны зарлалд энэхүү төсөл нь ownCloud-ийн үндсэн үүсгэн байгуулагчид, тэр дундаа ownCloud төслийн үүсгэн байгуулагч Frank Karlitschek-ийн оролцоотойгоор ownCloud-оос салаалан үүссэн fork гэж тодорхой дурдсан байдаг [12]. Иймээс Nextcloud болон ownCloud-ийг бүрэн хамааралгүй хоёр систем гэж үзэхээсээ илүү, нэг ижил гаралтай боловч дараа нь өөр архитектур, өөр бүтээгдэхүүний чиглэлээр хөгжсөн хоёр салбар гэж ойлгох нь зөв юм.

Архитектурын түвшинд Nextcloud нь хэрэглэгчийн хандалтын давхарга болон хадгалалтын давхаргын дунд байрлах зохион байгуулалттай. Browser, desktop/mobile client, мөн WebDAV зэрэг хандалтын сувгууд нэг Nextcloud сервер рүү холбогдож, цаанаа primary storage болон external storage-уудтай ажилладаг [13, 14]. Энэ нь хэрэглэгчийн хувьд файлууд нэг л цэгт төвлөрсөн мэт харагдах боловч системийн админ талд локал диск, object storage, SMB/CIFS, SFTP, WebDAV зэрэг өөр өөр backend-ийг холбох уян хатан боломж олгодог [14].

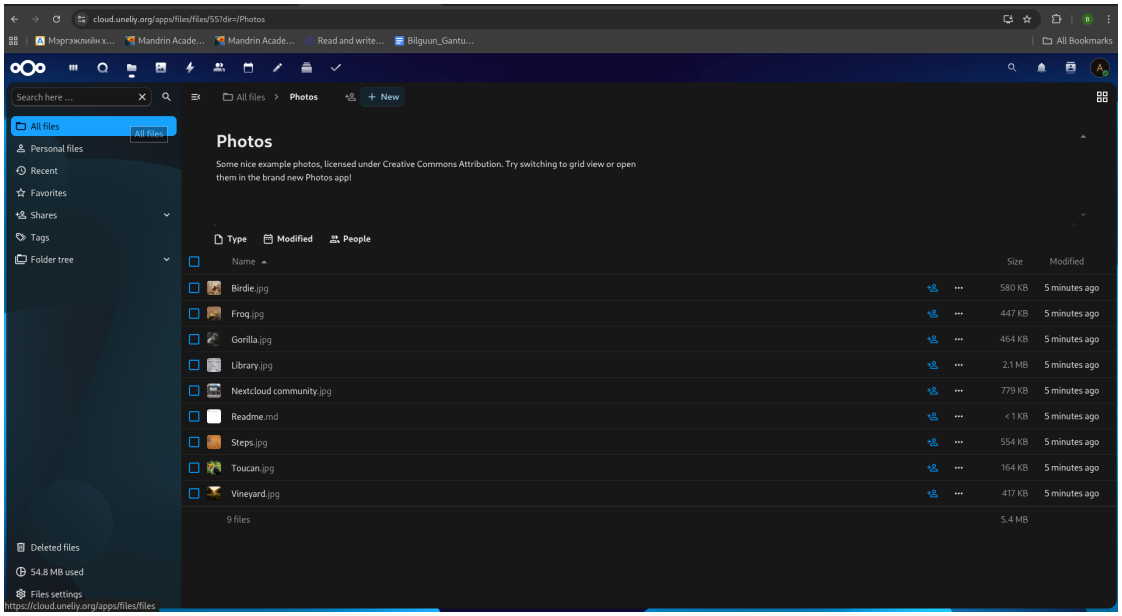


Зураг 1.3: Nextcloud-ийн хандалтын ба хадгалалтын давхаргын ерөнхий зураглал (эх сурвалж: Nextcloud Architecture Whitepaper) [13]

Зураг 1.3-т Nextcloud нь хэрэглэгчийн талын олон төрлийн интерфэйсийг доорх хадгалалтын эх үүсвэрүүдтэй холбож буй "зууч давхарга" хэлбэрээр харагдаж байна. Дээд хэсгийн browser, client, WebDAV нь хэрэглэгчид харагдах тал бол доод хэсгийн primary storage болон external storage нь бодит өгөгдөл байрлах орчин юм. Энэ зураг personal cloud системд дараах чухал санааг өгч байгаа юм: хэрэглэгчийн интерфэйсийг хадгалалтын физик байршлаас салгаж чадвал нэг талаас хэрэглээний туршлага жигд, нөгөө талаас дэд бүтцийн сонголт уян хатан хэвээр үлддэг.

Өгөгдлийн хяналтын хувьд Nextcloud нь Google Drive-тай харьцуулахад илүү шууд бөгөөд нарийн удирдлага өгдөг. Админ нь external storage mount-уудыг хэрэглэгч, бүлэг, mount option-оор нь ялган тохируулж, хуваалцах эсэх, read-only эсэх, encryption хэрэглэх эсэхийг хүртэл тус бүрээр нь зааж болно [14]. Үүн дээр File Access Control механизмыг ашиглавал дүрэм биелсэн үед тухайн файлыг upload хийх, өөрчлөх, устгах, татах, бүр клиентээр sync хийхийг хүртэл хориглох боломжтой [15]. Аюулгүй байдлын хувьд Nextcloud нь Server-Side Encryption болон End-to-End Encryption-ийг ялгаж авч үздэг; Server-Side Encryption нь файлыг сервер дээр болон external storage дээр хамгаалдаг бол End-to-End Encryption нь файлыг client талд шифрлэдэг учраас серверийн админ хүртэл агуулгыг тайлж үзэх боломжгүй байдаг [16]. Энэ нь personal

cloud системд хэрэгтэй маш чухал ялгаа бөгөөд “өгөгдөл миний сервер дээр байна” гэдэг нь дангаараа хангалтгүй, ямар түвшний хэн өгөгдлийг уншиж чадах вэ гэдгийг бас архитектурын түвшинд шийдэх шаардлагатайг харуулж байна.



Зураг 1.4: Docker орчинд ажиллуулсан Nextcloud-ийн веб интерфэйсийн дэлгэцийн зураг

Зураг 1.4 нь миний бие Nextcloud-ийг Docker орчинд амжилттай ажиллуулж турш-сан веб интерфэйсийн жишээ юм. Зурагт харагдаж буйгаар файлуудыг жагсаах, зураг хадгалах, хуваалцах, үндсэн navigation ашиглах зэрэг өдөр тутмын үүлэн хад-галалтын гол үйлдлүүд хэрэглэгчийн хувьд ойлгомжтой, бүрэн хэрэглэхүйц түвшинд ажиллаж байна. Энэ нь Nextcloud self-hosted шийдэл боловч хэрэглэгчийн туршла-гын хувьд public cloud үйлчилгээтэй төстэй мэдрэмж өгч чаддагийг баталж байна.

```
CONTAINER ID   NAME                          CPU %     MEM USAGE / LIMIT     MEM %
d92b16684685   nextcloud-aio-apache          0.03%     79.58MiB / 46.88GiB   0.17%
642a09d6396b   nextcloud-aio-nextcloud       0.03%     281.7MiB / 46.88GiB   0.59%
b02796f21b65   nextcloud-aio-imaginary       0.01%     225.6MiB / 46.88GiB   0.47%
ab9e15449795   nextcloud-aio-redis           0.27%     9.906MiB / 46.88GiB   0.02%
d53d07c76423   nextcloud-aio-database        3.91%     68.51MiB / 46.88GiB   0.14%
17fee3931d78   nextcloud-aio-whiteboard      0.01%     132.7MiB / 46.88GiB   0.28%
7cc917e82e7a   nextcloud-aio-notify-push     0.04%     5.34MiB / 46.88GiB    0.01%
134450ec6ec2   nextcloud-aio-talk            0.83%     126.9MiB / 46.88GiB   0.26%
07a7c84ccab0   nextcloud-aio-mastercontainer 0.42%     76.36MiB / 46.88GiB   0.16%
```

Зураг 1.5: Docker дээр ажиллуулсан Nextcloud AIO контейнерүүдийн санах ойн хэрэглээний дэлгэцийн зураг

Зураг 1.5-т миний Docker орчин дахь Nextcloud AIO deployment-ийн контейне-рүүдийн санах ойн хэрэглээг харуулсан. Тухайн зурагт байгаа `apache`, `nextcloud`,

`imaginary`, `redis`, `database`, `whiteboard`, `notify_push`, `talk`, `mastercontainer` зэрэг нийт контейнерүүдийн хэрэглээг нэгтгэн тооцоход ойролцоогоор 1006.6 MiB буюу бараг 1 GiB RAM ашиглаж байна. Энэ нь ердийн сервер орчинд боломжийн үзүүлэлт боловч Raspberry Pi зэрэг жижиг төхөөрөмж, эсвэл зөвхөн нэг хэрэглэгчийн хувийн хэрэглээнд зориулсан маш хөнгөн системийн хувьд өндөр шаардлага гэж үзэж болно. Өөрөөр хэлбэл Nextcloud нь боломж, өргөтгөл, collaboration талдаа хүчтэй боловч зөвхөн personal use-д чиглэсэн, бага нөөцтэй төхөөрөмж дээр ажиллах систем зохиох үед илүү минимал архитектур хэрэгтэй гэсэн дүгнэлтэд хүргэж байна.

Сэргээх болон ажиллагааны найдвартай байдлын талаас Nextcloud нь нэлээд боловсорсон механизмтай. Versions app нь файлын өмнөх хувилбаруудыг автоматаар хадгалж, хугацаа болон сул зайд тулгуурлан хуучин version-уудыг цэвэрлэдэг [17]. Trash bin нь устсан файлуудыг retention policy-аар тодорхой хугацаанд хадгалж, шаардлагатай үед буцаан сэргээх боломж өгдөг [18]. Харин файл эвдрэх, давхар өөрчлөлтөөс үүсэх асуудлыг бууруулахын тулд transactional file locking ашигладаг бөгөөд өндөр ачаалалтай орчинд Redis-ийг locking болон caching-д ашиглах нь ажиллагааг мэдэгдэхүйц тогтвортой болгодог [19, 20]. Гэхдээ эдгээр боломжийг найдвартай ажиллуулахын тулд database, cache, background job, TLS, backup policy, update lifecycle зэрэг ажиллагааны асуудлыг заавал урьдчилан төлөвлөх хэрэгтэй [21].

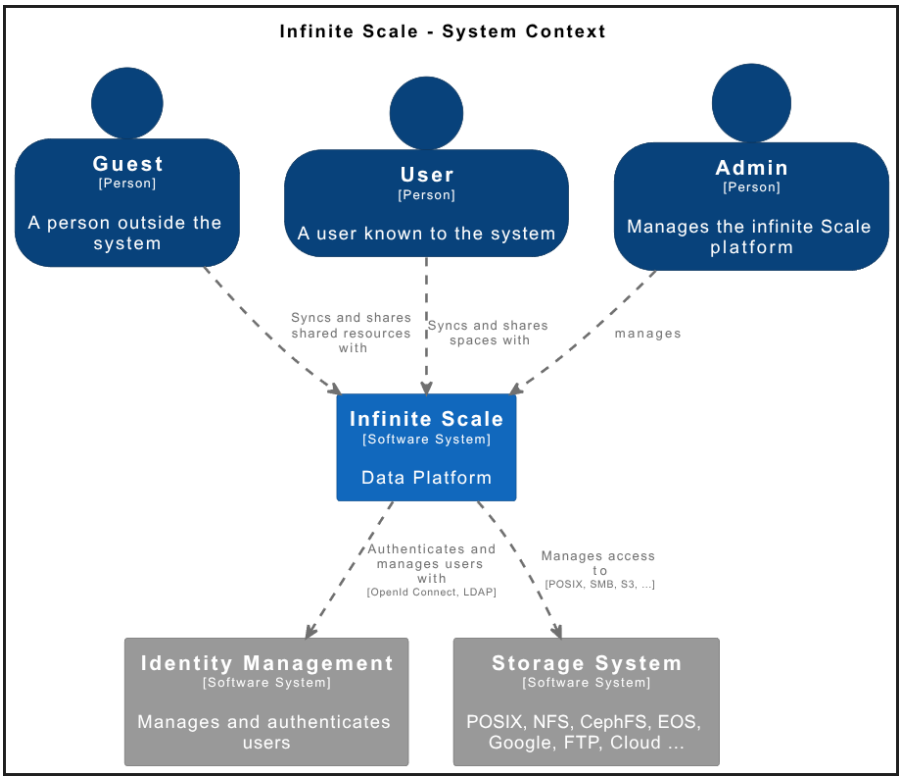
Дүгнэж хэлбэл Nextcloud нь personal cloud системийн үндсэн санаа болох “өгөгдөл өөрийн хяналтад байх ёстой” гэсэн шаардлагыг маш сайн хэрэгжүүлдэг жишээ юм. Гэвч миний Docker туршилтаас харахад ийм шийдэл нь олон боломж, олон үйлчилгээ дагалдуулж ирдэг тул нөөцийн хэрэглээ өсөх хандлагатай байна. Тиймээс энэхүү дипломын ажлын чиглэл нь Nextcloud-ийн өгөгдлийн хяналт, self-hosted зарчмыг хадгалах атлаа зөвхөн хувь хэрэглээнд зориулсан, жижиг төхөөрөмж дээр ч ажиллах илүү хөнгөн жинтэй cloud storage сервис боловсруулахад оршиж байна.

1.2.4 ownCloud Infinite Scale (oCIS): Микросервис Чиглэлийн Personal Cloud

ownCloud Infinite Scale (oCIS) нь self-hosted cloud storage-ийг уламжлалт ownCloud 10 загвараас өөр, илүү орчин үеийн аргаар шийдсэн платформ юм. ownCloud-ийн албан ёсны танилцуулгад Infinite Scale-ийг ownCloud-ийн cloud collaboration platform-ийн “next generation” гэж тодорхойлж, Go, Vue.js, three-tier architecture, cloud-native microservices зэрэг шинэ стек дээр тулгуурлан бүтээсэн гэж тайлбарласан байдаг [22].

Энэ утгаараа oCIS-ийг зүгээр нэг өөр бүтээгдэхүүн гэж бус, харин ownCloud-ийн дараагийн үеийн архитектурын шинэчлэл гэж ойлгож болно. ownCloud-ийн архитектурын баримт бичигт ч Infinite Scale-ийг data platform, unified data access, cloud data ecosystems гэсэн ойлголтуудыг төвд нь тавьж зохиосон гэж тайлбарласан байдаг [23]. Өөрөөр хэлбэл энэ нь зөвхөн "файл хадгалах сервер" биш, харин өгөгдөл, мета өгөгдөл, хуваалцах бодлого, identity integration-ийг салангид боловч уялдаа холбоотой үйлчилгээ болгон зохион байгуулахыг зорьсон систем юм.

oCIS-ийн гол онцлог нь микросервис чиглэлийн логиктой атлаа хэрэглээнд хэт хүндрэлтэй байхгүйгээр эхлүүлэхийг зорьдогт оршино. ownCloud-ийн тайлбарласнаар default тохиргоонд бүх built-in service-үүдийг нэг процесс дотор 'ocis server' командаар эхлүүлж болдог, харин шаардлага өсөх үед эдгээрийг өөр node-ууд руу салган байршуулж scale-out хийх боломжтой [23]. Энэ санаа нь онолын хувьд маш сонирхолтой: жижиг deployment-оор эхэлж, дараа нь proxy, graph, WebDAV, sharing, storage зэрэг хэсгүүдийг тус тусад нь өргөтгөж болно. Nextcloud-тай харьцуулахад энд "applications over one storage server" гэхээсээ илүү "cooperating services over a shared identity and storage model" гэсэн архитектурын сэтгэлгээ давамгайлж байна.



Зураг 1.6: ownCloud Infinite Scale-ийн system context зураг (эх сурвалж: ownCloud docs) [23]

Зураг 1.6 нь ownCloud-ийн албан ёсны system context дүрслэл бөгөөд oCIS-ийн ерөнхий байрлалыг маш ойлгомжтой харуулж байна. Дээд хэсэгт guest, user, admin гэсэн үндсэн оролцогчид байна. Тэдгээр нь төвд байрлах Infinite Scale data platform-той харилцаж, уг платформ нь доод талдаа identity management болон storage system гэсэн хоёр чухал гадаад тулгууртай холбогдож ажиллана. Энэ зураг нь oCIS-ийг "дан файл хадгалах сервер" гэхээсээ илүү, хэрэглэгч, эрхийн удирдлага, хадгалалт гэсэн гурван талыг холбодог платформ гэж ойлгоход тохиромжтой.

Функциональ концепцийн хувьд oCIS-ийн хамгийн сонирхолтой санаа нь "Spaces" юм. ownCloud-ийн баримтад spaces-ийг хэрэглэгчийн харах файлын логик орон зай гэж тайлбарладаг бөгөөд access нь roles-оор удирдагдана, мөн space бүр quota, manager, syncability зэрэг шинжтэй байж болно [23]. Энэ нь personal storage, project drive, group space, incoming share зэрэг өөр өөр төрлийн өгөгдлийг нэг ижил загвараар авч үзэх боломж олгодог. Ийм abstraction нь personal cloud-ийг цааш өргөтгөхөд ашигтай ч, анхны шатны хэрэгжүүлэлт дээр заавал бүхэлд нь дуурайх шаардлагагүй. Гэхдээ "файлын мод"-оос илүү "логик орон зай + эрхийн загвар" гэсэн сэтгэлгээг авч үзэх нь маш хэрэгтэй сургамж юм.

Аюулгүй байдлын хувьд oCIS нь үйлчилгээ бүр дээр тус тусад нь нэвтрэх эрх шалгахаас илүү gateway төвтэй зохион байгуулалтыг ашигладаг. ownCloud-ийн security баримтад бүх хүсэлт proxy service-ээр дамждаг тул бусад service-үүдийг firewall-ийн ард нууж, attack surface-ийг бууруулдаг гэж тайлбарласан [24]. Мөн OpenID Connect дээр суурилсан identity integration нь жижиг instance дээр дотоод IDM ашиглах, илүү бодит орчинд гадаад identity provider-тай холбох хоёр өөр зам нээж өгдөг [24]. Extensions/service-үүд нь standalone service байдлаар API-аар харилцдаг тул буруу ажилласан нэг extension бүх storage-д шууд нөлөөлөх эрсдэл бага байдаг [24]. Энэ нь policy isolation болон service isolation-ийг зэрэг авч үздэгээрээ Nextcloud-аас ялгаатай.

ownCloud өөрсдөө availability and scalability баримтдаа ийм төрлийн modern microservices-based software-ийг single эсвэл distributed instance байдлаар ажиллуулахдаа шийдвэрүүдийг маш нухацтай гаргах хэрэгтэй гэж онцолсон байдаг [25]. Архитектурын хувьд scale-out, container-based deployment, service separation нь маш хүчтэй боловч бодит ажиллагаан дээр service discovery, monitoring, logging, reverse proxy, identity integration, shared storage зэрэг олон шинэ асуудлыг хамтад нь авчирна [23, 25]. Тиймээс oCIS нь хувийн үүлэн хадгалалтын системийг ирээдүйд enterprise эсвэл multi-tenant чиглэлд өргөтгөхөд оновчтой санаа өгдөг ч, зөвхөн нэг хэрэглэгч эсвэл жижиг

төхөөрөмж дээр ажиллах хөнгөн системийн хувьд хэт их архитектурын давхарга болох эрсдэлтэй.

Дүгнэж хэлбэл ownCloud Infinite Scale нь personal cloud-ийн “дараагийн үе”-ийн загварыг харуулдаг: gateway төвтэй хамгаалалт, spaces дээр суурилсан access model, identity federation, scale-out чадвар. Харин энэхүү дипломын ажлын өнцгөөс хараахад эдгээр нь шууд хуулж хэрэгжүүлэх загвар биш, харин ирээдүйн өргөтгөлийн архитектурт ашиглаж болох гарын авлага юм. Өөрөөр хэлбэл oCIS-оос авах гол сургамж нь минимал хувийн системийг одоо хөнгөн байлгахын зэрэгцээ, цааш томрох үед service separation болон policy layer-уудыг хэрхэн нэмэхийг урьдчилан бодож загварчлах хэрэгтэй гэсэн санаа юм.

1.2.5 Нэгдсэн Харьцуулалт

Шалгуур	**Google Drive**	**Nextcloud**	**ownCloud oCIS**
Системийн зорилго	Хамтын ажиллагаа, синк, баримт бичгийн урсгалд хамгийн боловсорсон шийдэл.	Хувийн өгөгдлөө өөрийн сервер дээр төвлөрүүлж удирдахад хамгийн ойр шийдэл.	Томорч өргөжих self-hosted платформ руу чиглэсэн дараагийн үеийн шийдэл.
Архитектурын бүтэц	Маш хүчтэй distributed architecture-тай ч дотоод бүтэц нь хэрэглэгчид хаалттай.	Нэг сервер дээр ойлгомжтой ажиллуулж болдог ч нэмэлт сервисүүдтэй болох тусам хүндэрнэ.	Service-үүдээр сайн задарсан, өргөтгөхөд тохиромжтой ч анхнаасаа илүү нийлмэл.
Өгөгдлийн хяналт	Бодлогын түвшний хяналт сайн ч өгөгдлийн дэд бүтэц Google-ийн мэдэлд байна.	Өгөгдөл, storage, sharing, access rule-аа өөрөө удирдах боломж хамгийн өндөр.	Access model нь уян хатан боловч identity болон policy давхарга нь илүү төвөгтэй.
Аюулгүй байдал ба сэргээх боломж	Хувилбар, audit, recoverability маш сайн; enterprise түвшний найдвартай байдалтай.	Encryption, versioning, trash recovery сайн ч админы зөв тохиргооноос их хамаарна.	Gateway болон policy төвтэй хамгаалалт хүчтэй ч зөв ажиллуулахад илүү их тохиргоо шаарддаг.
Ашиглалт ба өргөтгөл	Хэрэглэхэд хамгийн хялбар, гэхдээ платформын хязгаарлалтаас бүрэн гарах боломжгүй.	Personal cloud-д шууд хэрэглэхэд хамгийн бодитой, харин нөөцийн хэрэглээ харьцангуй өндөр.	Ирээдүйд scale-out хийхэд хамгийн бэлэн, харин жижиг төхөөрөмж дээр хэт нарийн архитектур болж магадгүй.

Хүснэгт 1.1: Google Drive, Nextcloud, ownCloud oCIS системүүдийн нэгдсэн үнэлгээ

Дүгнэж хэлбэл, энэхүү дипломын ажлын хувьд **Nextcloud** нь хувийн өгөгдлөө өөрийн хяналт дор ажиллуулах хамгийн ойр жишээ болж байна. **oCIS** нь системийг цааш томруулж, илүү олон service-тэй болгох үед ямар архитектур хэрэгтэйг харуулж байна. Харин **Google Drive**-аас синк, өөрчлөлт хянах, файл сэргээх урсгалыг хэрхэн найдвартай зохион байгуулах санааг авах нь хамгийн тохиромжтой юм.

1.3 Ашиглах технологиуд

Энэхүү дипломын ажлын систем нь хэрэглэгчийн өөрийн сервер дээр ажиллах, хөнгөн бүтэцтэй байх, бага нөөцтэй орчинд ч ашиглагдах, мөн веб болон мобайл клиентээр хандах боломжтой байх шаардлагатай. Иймээс технологийн сонголтыг зөвхөн түгээмэл байдлаар нь биш, харин **нөөцийн хэрэглээ**, **deployment-ийн хялбар байдал**, **хөгжүүлэлтийн найдвартай байдал**, **хэрэглэгчид хүрэх интерфейс** гэсэн хэрэгцээтэй уялдуулан хийв.

Сервер талын хэрэгжүүлэлтэд **Rust** хэлийг сонгосон. Энэ системийн хамгийн чухал шаардлагын нэг нь жижиг сервер, хувийн компьютер, эсвэл нөөц багатай төхөөрөмж дээр ажиллах боломжтой байх явдал юм. Rust нь хөрвүүлэгддэг хэл учраас runtime орчны нэмэлт зардал багатай, нэг binary executable файл хэлбэрээр тараах боломжтой тул self-hosted системд маш тохиромжтой [26]. Өөрөөр хэлбэл хэрэглэгч серверийг ажиллуулахдаа олон dependency, interpreter, эсвэл хүнд runtime орчин суулгах шаардлага багасна.

Мөн Rust нь ownership, borrowing, lifetime зэрэг механизмаар дамжуулан санах ойтой холбоотой олон алдааг compile хийх үед илрүүлэх боломж олгодог [26]. Энэ нь файл хадгалах, хэрэглэгчийн эрх шалгах, upload/download урсгал боловсруулах серверийн хувьд найдвартай ажиллагаанд шууд нөлөөлнө. Async/await болон олон хүсэлтийг зэрэг боловсруулах чадвар нь хэдийгээр систем enterprise хэмжээний биш ч, нэгэн зэрэг хэд хэдэн файл татах, байршуулах, preview хийх үед тогтвортой ажиллах суурь болно. Иймээс Rust нь энэхүү ажлын "хөнгөн боловч найдвартай сервер" гэсэн зорилгод хамгийн сайн нийцэж байна.

Хэрэглэгчийн талд системийг хоёр сувгаар ашиглах шаардлагатай гэж үзсэн. Нэгдүгээрт, ямар ч төхөөрөмжөөс веб хөтчөөр хандах боломжтой **веб аппликейшн** хэрэгтэй. Хоёрдугаарт, iPhone болон iPad дээр илүү шууд, төхөөрөмжид нийцсэн хэрэглээ өгөх **iOS native аппликейшн** шаардлагатай. Иймээс клиент талыг веб болон iOS гэсэн хоёр хэлбэрээр хөгжүүлнэ.

iOS аппликейшн-ийг **Swift** болон **SwiftUI** ашиглан хөгжүүлнэ. Энэ сонголтын гол шалтгаан нь Apple төхөөрөмж дээр native интерфэйс, тогтвортой ажиллагаа, системийн боломжуудтай илүү сайн уялдах явдал юм. Файл сонгох, preview хийх, нэвтрэлт хадгалах, мобайл орчинд энгийн бөгөөд цэвэр интерфэйс гаргах зэрэгт native хөгжүүлэлт илүү тохиромжтой. Мөн энэхүү дипломын ажлын зорилго нь өргөн платформын framework туршихаас илүү, хэрэглэгчид шууд ашиглаж болох бодит мобайл клиент боловсруулах тул Swift/SwiftUI нь зөв сонголт болно.

Веб аппликейшн-ийг **Angular** framework ашиглан хөгжүүлнэ. Учир нь энэ системд login, file explorer, upload/download, user management, sharing зэрэг хоорондоо холбоотой олон дэлгэц, төлөв, формын урсгалууд байна. Angular нь ийм төрлийн бүтэцтэй веб системийг цэгцтэй зохион байгуулахад тохиромжтой. TypeScript суурьтай тул кодын бүтэц тодорхой, томроход арчлахад хялбар байна. Веб клиент нь хэрэглэгчдэд ямар ч нэмэлт суулгалтгүйгээр системд хандах үндсэн гарц болох бол iOS клиент нь өдөр тутмын мобайл хэрэглээг дэмжинэ.

Ингэснээр технологийн сонголт бүхэлдээ системийн зорилготой уялдаж байна: **Rust** нь хөнгөн, найдвартай, self-hosted сервер бүтээхэд; **Angular** нь удирдлагын болон файлын веб интерфэйсийг цэгцтэй хэрэгжүүлэхэд; **Swift/SwiftUI** нь iOS төхөөрөмж дээр энгийн, шууд хэрэглээ өгөхөд тус тус тохирч байна. Өөрөөр хэлбэл эдгээр технологиуд нь зөвхөн техник талаасаа боломжтой сонголт биш, харин энэхүү дипломын ажлын "хувийн, хөнгөн, өөрийн хяналттай үүлэн хадгалалтын систем" гэсэн хэрэгцээнд нийцсэн сонголтууд юм.

1.4 Бүлгийн дүгнэлт

Энэ бүлэгт үүлэн хадгалалтын системийн үндсэн ойлголт, хөгжлийн чиг хандлага, мөн ижил төст платформуудын архитектурын санааг судаллаа. Google Drive, Nextcloud, ownCloud oCIS системүүдийг харьцуулж үзэхэд public cloud шийдлүүд нь найдвартай ажиллагаа, синк, сэргээх урсгалаараа давуу боловч өгөгдлийн бүрэн хяналт хэрэглэгчийн талд байдаггүй нь харагдлаа. Харин self-hosted чиглэлийн шийдлүүд нь өгөгдлийн өмчлөл, хандалтын хяналтыг илүү сайн хангаж байгаа ч суулгалт, тохиргоо, нөөцийн хэрэглээний хувьд илүү анхаарал шаарддаг байна.

Иймээс энэхүү дипломын ажлын хувьд хөнгөн бүтэцтэй, өөрийн сервер дээр ажиллах, хэрэглэгч өгөгдлөө өөрөө хянах боломжтой personal cloud систем боловсруулах чиглэл нь хамгийн оновчтой гэж дүгнэв. Мөн серверт Rust, веб клиентэд Angular, iOS клиентэд Swift/SwiftUI ашиглах нь системийн хэрэгцээтэй нийцэж байгааг тодорхойлов. Дараагийн бүлэгт энэхүү дүгнэлтэд тулгуурлан системийн шаардлага, хэрэглэгчийн үүрэг, үндсэн үйл ажиллагааг нарийвчлан тодорхойлно.

БҮЛЭГ 2

Хувийн үүлэн хадгалалтын системийн шинжилгээ

2.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй

Хэрэглэгч үндсэн серверийн программыг өөрийн хүссэн серверийн компьютер дээр ажиллуулах бөгөөд сүлжээний хандалтын асуудлыг өөрөө шийдвэрлэнэ. Серверийн програмыг ажиллуулсны дараа хэрэглэгч системийн admin interface-д нэвтэрч анхны администратор (admin) хэрэглэгчийг үүсгэнэ. Мөн энэ үед системийн үндсэн үүлэн хадгалалтын хавтсыг (storage directory) зааж өгнө.

Анхны тохиргоо хийгдсэний дараа хэрэглэгч веб аппликейшн эсвэл iOS гар утасны аппликейшн ашиглан системд нэвтрэх боломжтой болно. Администратор хэрэглэгч нь системд шинэ хэрэглэгч үүсгэх, хэрэглэгчийн мэдээллийг удирдах, мөн хэрэглэгчдэд тодорхой хавтас болон файл дээр read эсвэл edit эрх олгох боломжтой байна.

Администратор хэрэглэгч нь системийн хадгалалтын орчинд дурын хавтас болон файлыг үүсгэх, засах, устгах, мөн файлуудыг download болон upload хийх бүрэн эрхтэй байна.

Энгийн хэрэглэгч нь администратороос олгосон эрхийн хүрээнд системд хадгалагдсан файлууд болон хавтаснуудтай ажиллах боломжтой бөгөөд файлыг харах, татаж авах, эсвэл засварлах зэрэг үйлдлүүдийг зөвшөөрөгдсөн хэмжээнд гүйцэтгэнэ.

2.2 Функциональ шаардлага

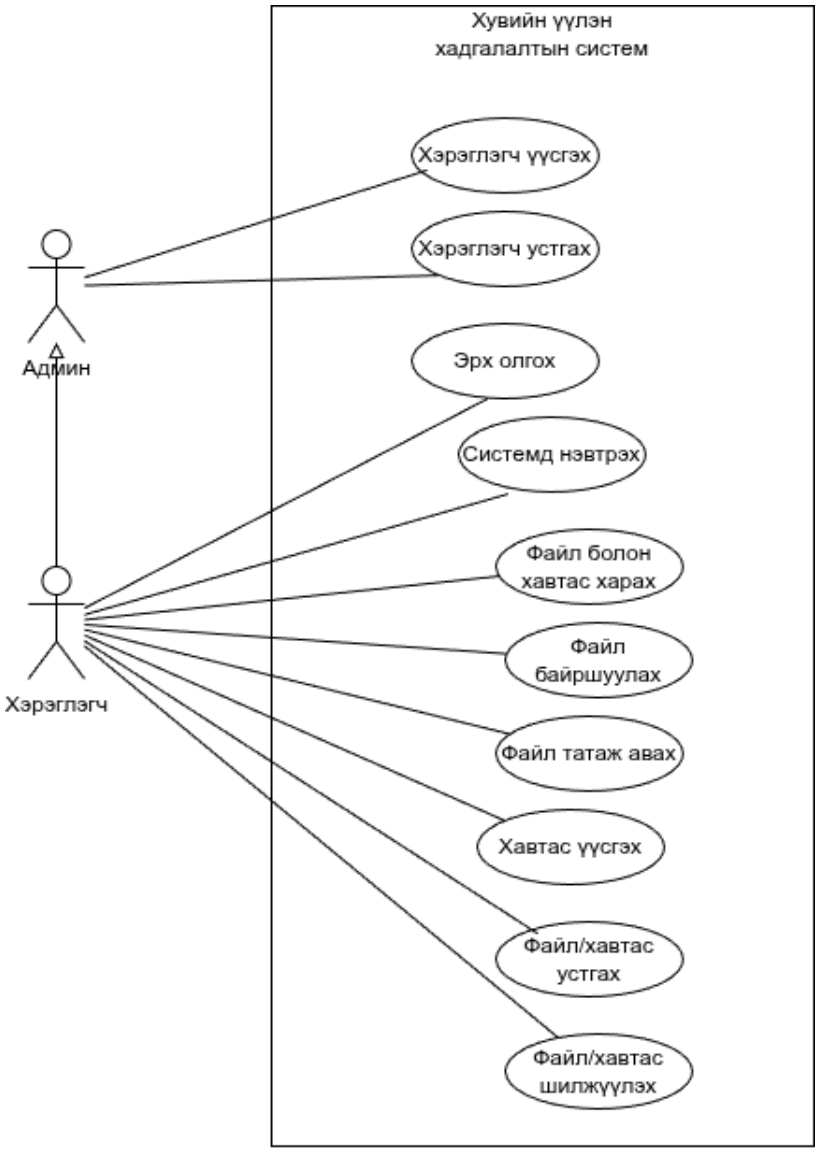
1. Серверийг анх ажиллуулах үед анхны админ хэрэглэгчийг үүсгэх боломжтой байх.
2. Үүлэн хадгалалтад ашиглах үндсэн хадгалалтын хавтсыг (storage folder) сонгон тохируулах боломжтой байх.
3. Хэрэглэгч веб аппликейшн болон iOS гар утасны аппликейшн ашиглан системд нэвтрэх боломжтой байх.
4. Админ шинэ хэрэглэгч үүсгэх боломжтой байх.
5. Админ хэрэглэгчийн нууц үгийг өөрчлөх болон хэрэглэгчийг устгах боломжтой байх.
6. Админ хэрэглэгчдэд тодорхой файл болон хавтас дээр read эсвэл edit эрх олгох боломжтой байх.
7. Хэрэглэгч өөрийн edit эрхтэй файл болон хавтас дээр бусад хэрэглэгчдэд эрх олгох боломжтой байх.

8. Хэрэглэгч системд хадгалагдсан файл болон хавтасны бүтцийг харах боломжтой байх.
9. Хэрэглэгч сервер рүү файл байршуулах (upload) боломжтой байх.
10. Хэрэглэгч системд хадгалагдсан файлыг татаж авах (download) боломжтой байх.
11. Хэрэглэгч шинэ хавтас үүсгэх боломжтой байх.
12. Хэрэглэгч файл болон хавтсын нэрийг өөрчлөх боломжтой байх.
13. Хэрэглэгч файл болон хавтсыг устгах боломжтой байх.
14. Хэрэглэгч файл болон хавтсыг өөр хавтас руу шилжүүлэх боломжтой байх.
15. Хэрэглэгч дараах төрлийн файлуудыг preview хийж харах боломжтой байх. Зураг (jpg, png, gif, webp), Text (txt, json, csv, md), Documents (pdf), Audio (mp3), Video (mp4)

2.2.1 Usecase диаграмм

Дараах юзкейс диаграммд системийн үндсэн 2 хэрэглэгч болох admin болон энгийн хэрэглэгч гэсэн 2 оролцогчийг дүрслэв. Admin хэрэглэгч нь энгийн хэрэглэгчээс удамшиж байгаа ба энгийн хэрэглэгчийн бүх usecase-ийг хийх чадамжтай юм.

Системийн эхний setup хийх хэсэг буюу admin хэрэглэгчийг үүсгэх болон үндсэн хавтасыг сонгох хэсгийг энэхүү usecase диаграмм дээр оруулаагүй болно.



Зураг 2.1: Usecase диаграм

2.2.2 Юзкейсийн тодорхойлолт

Хэрэглэгч үүсгэх (UC-001)

Usecase ID	UC-001
Usecase name	Хэрэглэгч үүсгэх
Actor(s)	Админ
Description	Админ хэрэглэгч үүлэн хадгалалтын системд шинэ хэрэглэгч үүсгэх боломжтой байна.
Trigger	Үүлэн хадгалалтын системийг өөр хүмүүстэй хамт ашиглах шаардлага гарсан тохиолдолд
Preconditions	• Админ системд нэвтэрсэн байх.
Postconditions	• Шинэ хэрэглэгч үүссэн бөгөөд системд нэвтрэх нэр болон нууц үгээ ашиглан нэвтрэх боломжтой байх.
Main success scenario	1. Админ хэрэглэгчийн менежмент хэсэг рүү орно. 2. Шинэ хэрэглэгч үүсгэх товчийг дарна. 3. Хэрэглэгчийн нэвтрэх нэр болон ашиглах нууц үгийг оруулна. 4. Үүсгэх товчийг дарна. 5. Хэрэглэгч амжилттай үүсэж хэрэглэгчийг жагсаалт дээр харагдана.
Alternative flows	
Exception flows	Хэрэглэгчийн нэр давхцвал алдаа гарна.
Acceptance criteria	Шинэ хэрэглэгч амжилттай бүртгэгдсэн байх.
Special requirements	

Хүснэгт 2.1: UC-001 usecase-ийн тодорхойлолт

Хэрэглэгч устгах (UC-002)

Usecase ID	UC-002
Usecase name	Хэрэглэгч устгах
Actor(s)	Admin
Description	Admin систем дээр бүртгэлтэй байгаа хэрэглэгчдийг устгах боломжтой байна.
Trigger	Устгах шаардлагатай хэрэглэгч гарсан.
Preconditions	• Admin системд нэвтэрсэн байх.
Postconditions	• Хэрэглэгч амжилттай устгагдсан байх. • Хэрэглэгчийн мэдээлэл өгөгдлийн сангаас устсан байх.
Main success scenario	1. Admin хэрэглэгчийн менежмент хэсэг рүү орно. 2. Хэрэглэгчийн жагсаалтаас устгах хэрэглэгчээ сонгож устгах товч дарна. 3. Баталгаажуулах цонх гарч ирж баталгаажуулна. 4. Хэрэглэгчийг устгана.
Alternative flows	
Exception flows	
Acceptance criteria	Хэрэглэгч устсан байх. Тухайн хэрэглэгчээр нэвтрэх боломжгүй болсон байх.
Special requirements	Хүсэлт аюулгүй сувгаар дамжих.

Хүснэгт 2.2: UC-002 usecase-ийн тодорхойлолт

Эрх олгох (UC-003)

Usecase ID	UC-003
Usecase name	Эрх олгох
Actor(s)	Админ, Хэрэглэгч
Description	Хэрэглэгч өөрийн эрхтэй хавтас болон файл дээр бусад хэрэглэгч нарт эрх олгох боломжтой. Админ өөрийн хүссэн хавтас болон файл дээр хүссэн хэрэглэгчдээ эрх өгөх боломжтой.
Trigger	Бусад хэрэглэгч нарт ямар нэгэн хавтас болон файл хуваалцах шаардлага гарсан тохиолдолд
Preconditions	• Тухайн хэрэглэгч нэвтэрсэн байх. • Хуваалцах гэж буй хавтас болон файл тухайн хэрэглэгчийнх байх.
Postconditions	• Эрхийг авч буй хэрэглэгч дээр эрх нэмэгдсэн байх.
Main success scenario	1. Хэрэглэгч үүлэн хадгалалт дотроо эрх олгох гэж буй файл болон хавтсан дээрээ очно. 2. Хуваалцах товч дээр дарна. 3. Хэрэглэгч сонгоод баталгаажуулах товч дарна.
Alternative flows	
Exception flows	
Acceptance criteria	Хэрэглэгч дээр эрх үүссэн байх.
Special requirements	Хэрэглэгч тухайн файл эсвэл хавтасны эзэн биш бол хуваалцах товч харагдах ёсгүй. Мөн эзэн мөн эсэх шалгалтыг сервер талд гүйцэтгэнэ.

Хүснэгт 2.3: UC-003 usecase-ийн тодорхойлолт

Системд нэвтрэх (UC-004)

Usecase ID	UC-004
Usecase name	Системд нэвтрэх
Actor(s)	Админ, хэрэглэгч
Description	Үүлэн хадгалалтын үндсэн систем рүү нэвтрэх
Trigger	Хувийн үүлэн хадгалалтын системийг ашиглах шаардлага гарсан.
Preconditions	• Тухайн нэвтрэх гэж буй хэрэглэгчийн мэдээлэл системд бүртгэгдсэн байх.
Postconditions	• Системд амжилттай нэвтэрч өөрт хамааралтай хавтас болон файлуудыг харах
Main success scenario	1. Хэрэглэгч нэвтрэх хуудсыг нээнэ. 2. Нэвтрэх нэр болон нууц үгийг оруулаад нэвтрэх товчийг дарна. 3. Амжилттай тохиолдолд хэрэглэгчийг нэвтрүүлнэ. 4. Хэрэглэгч үндсэн хуудас руу үсэрнэ.
Alternative flows	
Exception flows	Нэвтрэх нэр болон нууц үг буруу бол алдааны мессеж харуулна.
Acceptance criteria	Хэрэглэгч системд амжилттай нэвтэрсэн байна.
Special requirements	

Хүснэгт 2.4: UC-004 usecase-ийн тодорхойлолт

Файл болон хавтас харах (UC-005)

Usecase ID	UC-005
Usecase name	Файл болон хавтас харах (Browse хийх)
Actor(s)	Хэрэглэгч
Description	Хадгалагдаж буй файлууд болон хавтасаар зочилж, сонирхох
Trigger	Файл эсвэл хавтас хайх шаардлага гарсан эсвэл ямар файл болон хавтаснууд байгааг сонирхсон.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх.
Postconditions	•
Main success scenario	1. Хэрэглэгч хүссэн хавтас болон файлуудаа харна.
Alternative flows	Хэрэглэгч хайлтын хэсгээс файл болон хавтсыг нэрээр хайх эсвэл файлын төрлөөр хайх боломжтой.
Exception flows	
Acceptance criteria	Хэрэглэгч эрхтэй файлууд болон хавтсуудаа үзсэн.
Special requirements	Хэрэглэгчид зөвхөн эрхтэй хавтас болон файлуудыг харуулна.

Хүснэгт 2.5: UC-005 usecase-ийн тодорхойлолт

Файл байршуулах (UC-006)

Usecase ID	UC-006
Usecase name	Файл байршуулах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн owner болон edit эрхтэй хавтас дотор файл байршуулах боломжтой.
Trigger	Хэрэглэгч өөрийн үүлэн хадгалалтын төхөөрөмж дээр файл байршуулах шаардлага гарах.
Preconditions	• Хэрэглэгч нэвтэрсэн байх.
Postconditions	• Файл системийн файл системд байршсан байх.
Main success scenario	1. Хэрэглэгч үндсэн хуудас дээрх файл системээр явж файл байршуулах хавтас руугаа орно. 2. Файл байршуулах товчийг сонгоно. 3. Файл сонгох цонх гарч ирж хэрэглэгч нэг болон түүнээс их файл сонгоно. 4. Файлуудыг upload хийгдэж буй явц харагдана.
Alternative flows	
Exception flows	Файлыг upload хийх явцад сүлжээ тасрах магадлалтай.
Acceptance criteria	Тухайн файл сервер дээр хадгалагдах
Special requirements	

Хүснэгт 2.6: UC-006 usecase-ийн тодорхойлолт

Файл татаж авах (UC-007)

Usecase ID	UC-007
Usecase name	Файл татаж авах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн read, edit, owner эрхтэй файлыг татах
Trigger	Хэрэглэгчид файлыг татах шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх.
Postconditions	• Файл хэрэглэгчийн төхөөрөмж дээр татагдсан байх
Main success scenario	1. Хэрэглэгч үндсэн хуудас дээрх файл системээр явж татахыг хүссэн файлынхаа хавтас руу орно. 2. Татах товчийг дарна. 3. Файл татах явц харагдана.
Alternative flows	
Exception flows	Файлыг татах явцад сүлжээ тасрах магадлалтай
Acceptance criteria	Файл хэрэглэгчийн төхөөрөмж дээр татагдсан байх
Special requirements	

Хүснэгт 2.7: UC-007 usecase-ийн тодорхойлолт

Хавтас үүсгэх (UC-008)

Usecase ID	UC-008
Usecase name	Хавтас үүсгэх
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч шинэ хавтас үүсгэж өөрийн файлуудыг зохион байгуулна.
Trigger	Хэрэглэгч шинэ хавтас үүсгэх шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч хавтас үүсгэх эрхтэй байх.
Postconditions	• Шинэ хавтас системд амжилттай үүссэн байна.
Main success scenario	1. Хэрэглэгч файл системийн интерфэйс дээр байрлах хавтас үүсгэх товчийг дарна. 2. Шинэ хавтасны нэрийг оруулна. 3. Систем шинэ хавтсыг үүсгэж харуулна.
Alternative flows	1. Хэрэглэгч хавтасны нэрийг өөрчлөх боломжтой.
Exception flows	Ижил нэртэй хавтас аль хэдийн үүссэн байж болно.
Acceptance criteria	Шинэ хавтас файл системд харагдаж байх.
Special requirements	

Хүснэгт 2.8: UC-008 usecase-ийн тодорхойлолт

Файл/хавтас устгах (UC-009)

Usecase ID	UC-009
Usecase name	Файл/хавтас устгах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн эрхтэй файл эсвэл хавтсыг системээс устгана.
Trigger	Хэрэглэгч файл эсвэл хавтас устгах шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч тухайн файл эсвэл хавтсыг устгах эрхтэй байх.
Postconditions	• Файл эсвэл хавтас системээс устсан байна.
Main success scenario	1. Хэрэглэгч устгахыг хүссэн файл эсвэл хавтсыг сонгоно. 2. Устгах товчийг дарна. 3. Систем баталгаажуулах цонх харуулна. 4. Хэрэглэгч баталгаажуулна. 5. Систем файл эсвэл хавтсыг устгана.
Alternative flows	1. Хэрэглэгч устгах үйлдлийг цуцалж болно.
Exception flows	Файл ашиглагдаж байгаа үед устгах боломжгүй байж болно.
Acceptance criteria	Файл эсвэл хавтас системээс бүрэн устсан байх.
Special requirements	

Хүснэгт 2.9: UC-009 usecase-ийн тодорхойлолт

Файл/хавтас шилжүүлэх (UC-010)

Usecase ID	UC-010
Usecase name	Файл/хавтас шилжүүлэх
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч файл эсвэл хавтсыг өөр хавтас руу шилжүүлнэ.
Trigger	Хэрэглэгч файл эсвэл хавтасны байршлыг өөрчлөх шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч тухайн файл эсвэл хавтсыг өөрчлөх эрхтэй байх.
Postconditions	• Файл эсвэл хавтас шинэ байршилд шилжсэн байна.
Main success scenario	1. Хэрэглэгч шилжүүлэх файл эсвэл хавтсыг сонгоно. 2. Шилжүүлэх үйлдлийг сонгоно. 3. Зорилтот хавтсыг сонгоно. 4. Систем файл эсвэл хавтсыг шинэ байршилд шилжүүлнэ.
Alternative flows	1. Хэрэглэгч шилжүүлэх үйлдлийг цуцалж болно.
Exception flows	Зорилтот хавтас олдохгүй байх магадлалтай.
Acceptance criteria	Файл эсвэл хавтас шинэ хавтас дотор харагдаж байх.
Special requirements	

Хүснэгт 2.10: UC-010 usecase-ийн тодорхойлолт

2.3 Функциональ бус шаардлага

1. Серверийн программ нь Linux, Windows, MacOS үйлдлийн системүүд дээр ажиллах боломжтой байх.

2. Хэрэглэгчийн client аппликейшн нь web болон iOS платформ дээр ажиллах боломжтой байх.
3. Web аппликейшн нь Chrome, Firefox, Safari болон Edge браузеруудын сүүлийн хоёр хувилбар дээр зөв ажиллах боломжтой байх.
4. Хэрэглэгч болон серверийн хооронд дамжих бүх өгөгдөл HTTPS (TLS) протокол ашиглан шифрлэгдсэн байх.
5. Хэрэглэгчийн нууц үгийг сервер дээр plaintext хэлбэрээр хадгалахгүй бөгөөд аюулгүй hash алгоритм ашиглан хадгалдаг байх.
6. Систем нь нэг сервер дээр олон хэрэглэгчийн зэрэг хандалтыг дэмжих боломжтой байх.
7. Файл байршуулах болон татах үйлдлийн явцыг хэрэглэгчийн интерфэйс дээр харуулах боломжтой байх.
8. Систем нь хэрэглэгч зөвхөн өөрийн файл болон өөрт олгогдсон эрхтэй өгөгдөлд хандах боломжийг хангах байх.
9. Файл дамжуулах явцад өгөгдлийн бүрэн бүтэн байдлыг шалгах механизм (checksum эсвэл hash) ашигладаг байх.

2.4 Ажлын хүрээ

2.4.1 Scope (Хамрах хүрээ)

Энэхүү төслийн хүрээнд дараах програм хангамжийн бүрэлдэхүүнүүдийг хөгжүүлнэ:

1. **Хувийн үүлэн хадгалалтын үндсэн серверийн программ (Main Server Application):** Үндсэн серверийн программ нь хэрэглэгчийн компьютерт суурилж, өгөгдлийг хадгалах, төхөөрөмж бүртгэх, холболтыг удирдах, файл дамжуулалтыг гүйцэтгэх үүрэгтэй байна.
2. **iOS үйлдлийн системд зориулсан мобайл клиент аппликейшн:** iOS мобайл аппликейшн нь хувийн үүлэн хадгалалтын серверийн үндсэн үйлдлүүдийг хийх боломжтой байна. Нэвтрэх, файл болон хавтас үзэх, файл татах, файл байршуулах гэх мэт дээр оруулсан үйлдлүүдийг хийнэ.
3. **Веб апп:** iOS апп-тай адилхан бүх үйлдлийг хийх боломжтой байна. Үндсэн үйлдлээс гадна. Хамгийн эхний тохигооны алхам буюу админ хэрэглэгч үүсгэх болон үндсэн хавтас сонгох алхамууд нэмэгдэж хийгдэнэ.

2.4.2 Out of Scope (Хийхгүй зүйлс)

1. **Android мобайл аппликейшн:**

Android үйлдлийн системд зориулсан мобайл аппликейшн энэхүү төслийн хүрээнд хөгжүүлэхгүй. Төслийн хүрээнд нэг платформ дээр төвлөрч, native хөгжүүлэлтийг гүнзгий түвшинд хийх зорилгоор iOS платформыг сонгосон болно.

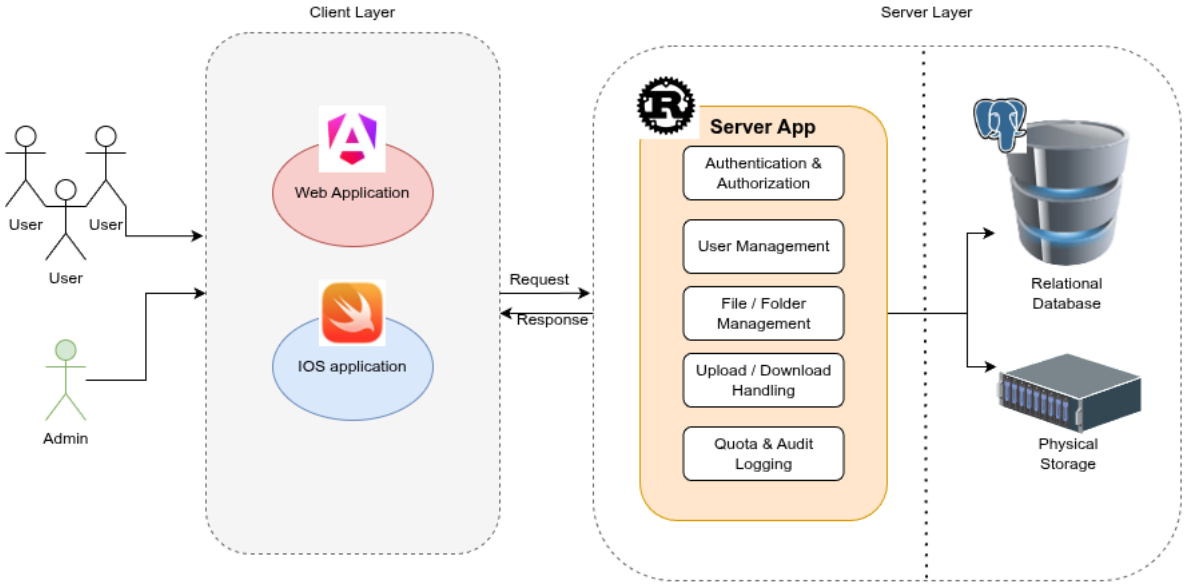
2.5 Бүлгийн дүгнэлт

БҮЛЭГ 3

Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт

3.1 Системийн архитектур

Дараах зурагт хувийн үүлэн хадгалалтын системийн архитектур диаграммыг харуулав. Энгийн хэрэглэгч болон admin гэсэн 2 төрлийн хэрэглэгч байх агаад бүх хэрэглэгчид веб олон IOS апп ашиглан системтэй харьцана. Веб апп-ийг Angular, IOS апп-ийг Swift технелогиудыг ашиглан хөгжүүлнэ. Тухайн хэрэглэгчийн апп-ууд нь сервертэй HTTPS холболтоор харьцаж зорилтот үйлдлүүдийг гүйцэтгэдэг байна. Сервер апп нь хэрэглэгчдийн нэвтрэлт, баталгаажуулалт, хэрэглэгчийн менежмент, файл менежмент, файл татах болон байршуулах үйлдлүүдийг хийнэ. Системийн нийт өгөгдлийг Relational өгөгдлийн бааз болон серверийн File system-д хадгална. Тухайн үүлэн хадгалалт дээр байрших файлууд болон хавтаснуудын мета өгөгдлүүд нь өгөгдлийн бааз дээр хадгалагдана.



Зураг 3.1: Системийн архитектур

3.2 Зохиомжийн үндсэн компонентууд

Системийн зохиомжид дараах үндсэн компонентуудыг тодорхойлсон.

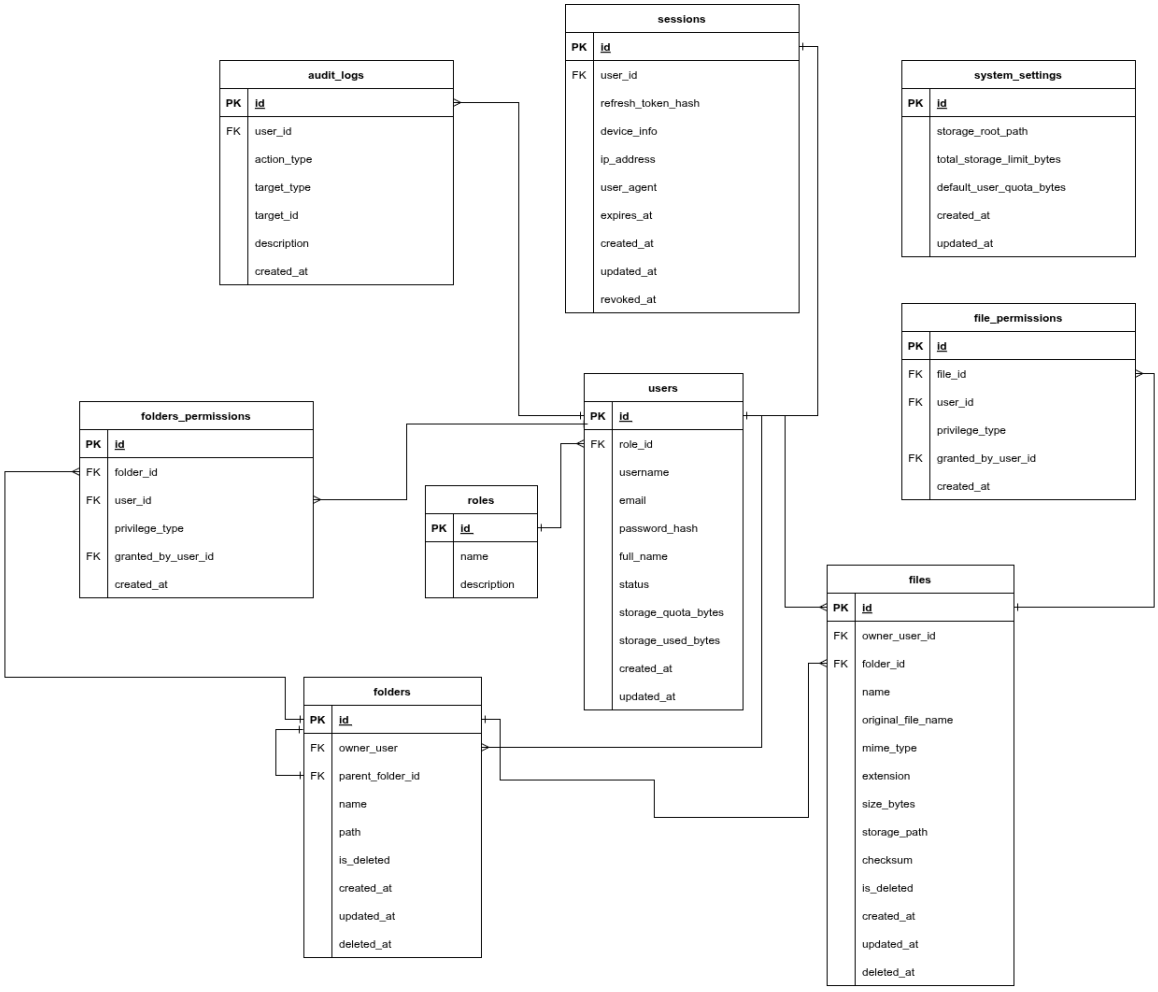
1. **Хэрэглэгчийн удирдлагын модуль** – нэвтрэх, нууц үг шалгах, session/token удирдах.
2. **Файл удирдлагын модуль** – хавтас үүсгэх, файл байршуулах, татах, нэр өөрчлөх, устгах.
3. **Хуваалцах эрхийн модуль** – хэрэглэгч болон бүлгийн түвшинд read/edit эрх олгох.

4. **Storage abstraction** – локал диск дээрх өгөгдөл болон өгөгдлийн сангийн мета мэдээллийг уялдуулах.

3.3 Өгөгдлийн ерөнхий схем

Зурагт үзүүлсэн энэхүү өгөгдлийн сангийн загвар (ERD) нь хувийн үүлэн хадгалалтын системийн үндсэн өгөгдлийн бүтэц болон тэдгээрийн харилцан хамаарлыг тодорхойлж байна. Систем нь хэрэглэгч (*users*) болон үүрэг (*roles*)-ийн удирдлагыг дэмжихийн зэрэгцээ хавтас (*folders*) болон файл (*files*)-ын шаталсан бүтэц бүхий логик хадгалалтыг хэрэгжүүлнэ. Хавтас нь *parent_folder_id* талбараар дамжин өөртэйгээ рекурс хамаарал үүсгэж, мод хэлбэрийн бүтэц бүрдүүлдэг бол файл нь тодорхой хавтсанд хамаарах байдлаар зохион байгуулагдана. Файлын бодит өгөгдөл нь физик хадгалалтын орчинд байрлах бөгөөд *storage_path* талбараар заагдана.

Системд хэрэглэгчийн хандалтын эрхийг удирдах зорилгоор *folder_permissions* болон *file_permissions* хүснэгтүүдийг ашигласан ба эдгээр нь хэрэглэгч бүрт *read* болон *edit* төрлийн эрхийг оноох боломжийг олгоно. Файл болон хавтас бүр нь *owner_user_id* талбараар эзэмшигчийг тодорхойлж, эзэмшигч нь бүрэн хяналттай байх зарчим хэрэгжинэ. Мөн хэрэглэгчийн нэвтрэлтийн сессийг *sessions* хүснэгтээр дамжуулан удирдаж, refresh token-ийн хэш, төхөөрөмжийн мэдээлэл болон хугацааны хязгаарыг хадгална. Үүнээс гадна системийн үйл ажиллагааг бүртгэх зорилгоор *audit_logs* хүснэгтийг ашиглаж, хэрэглэгчийн хийсэн үйлдэл, зорилтот объект болон хугацааг тэмдэглэн хадгална.

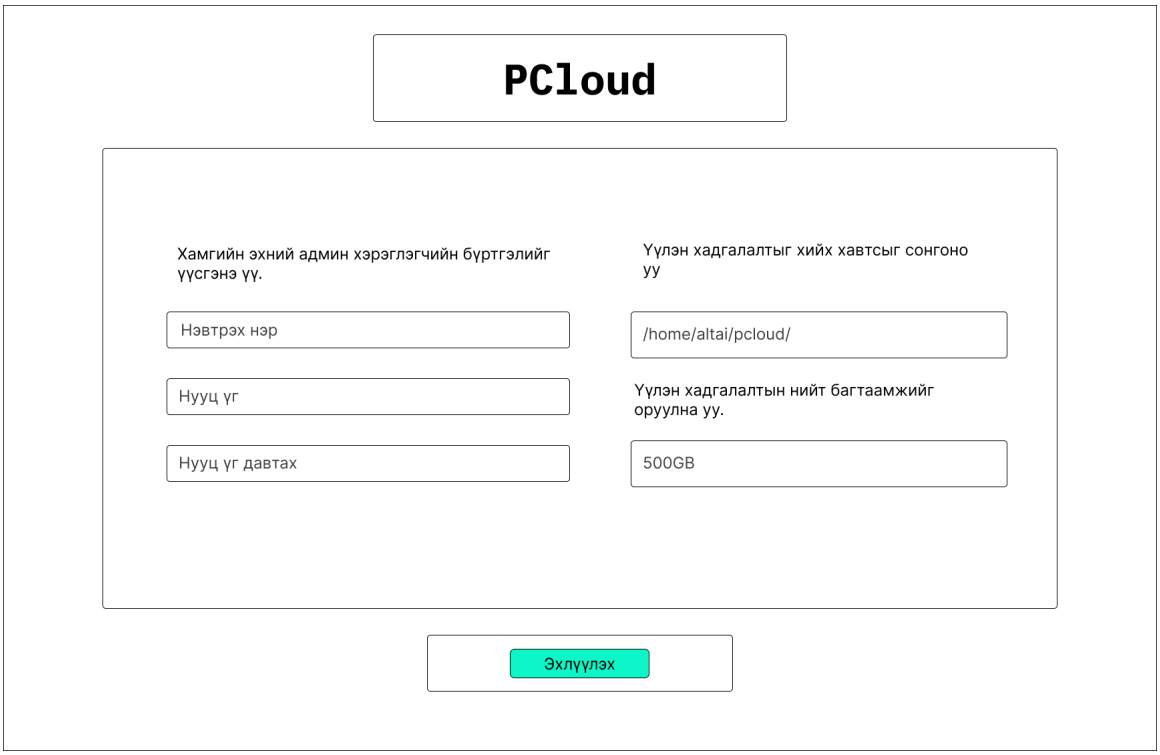


Зураг 3.2: Хувийн үүлэн хадгалалтын системийн ERD

3.4 Системийн прототип

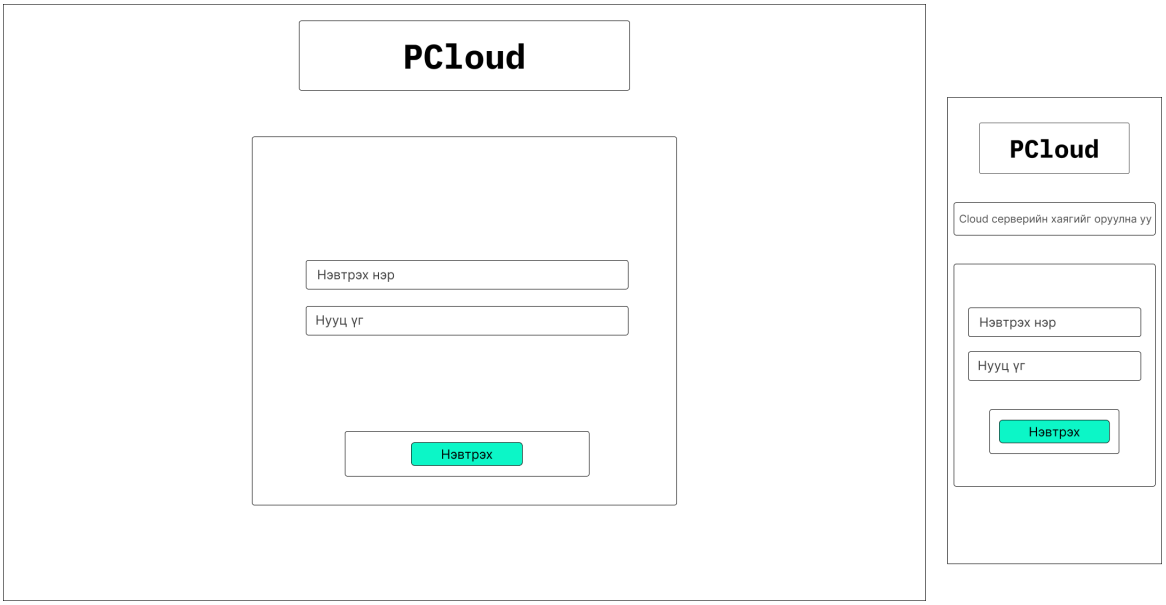
Системийн ерөнхий интерфейсийн прототипийг Figma орчинд бэлтгэсэн. Прототип нь эхний тохиргоо, нэвтрэх хуудас, энгийн хэрэглэгчийн нүүр хуудас, админ хэрэглэгчийн нүүр хуудас гэсэн үндсэн урсгалуудыг хамарсан.

Системийг анх ажиллуулах үед админ хэрэглэгч болон storage folder-ийн үндсэн тохиргоог авах дэлгэц нээгдэнэ. Энэ алхам нь self-hosted системийн хамгийн чухал эхний шат бөгөөд өгөгдөл хаана хадгалагдахыг тодорхойлно.



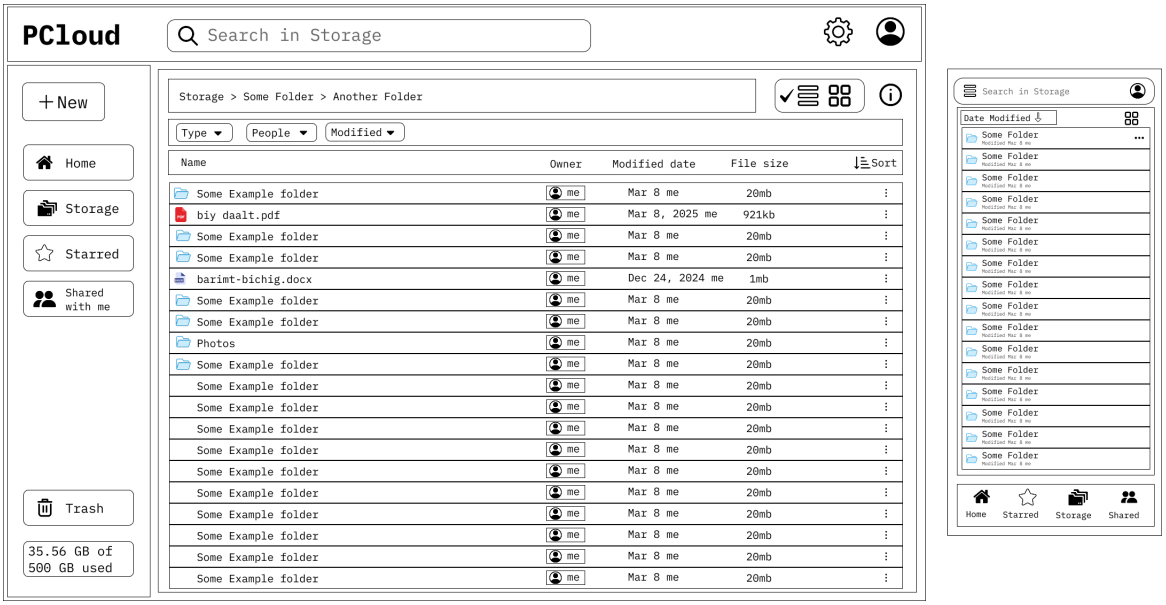
Зураг 3.3: Анхны тохиргооны дэлгэц

Нэвтрэх дэлгэц нь хэрэглэгчийн нэр, нууц үг, шаардлагатай үед серверийн хаяг оруулах талбаруудтай. Энэ загвар нь веб болон мобайл клиентээс нэг ижил backend-т холбогдох боломж олгодог.



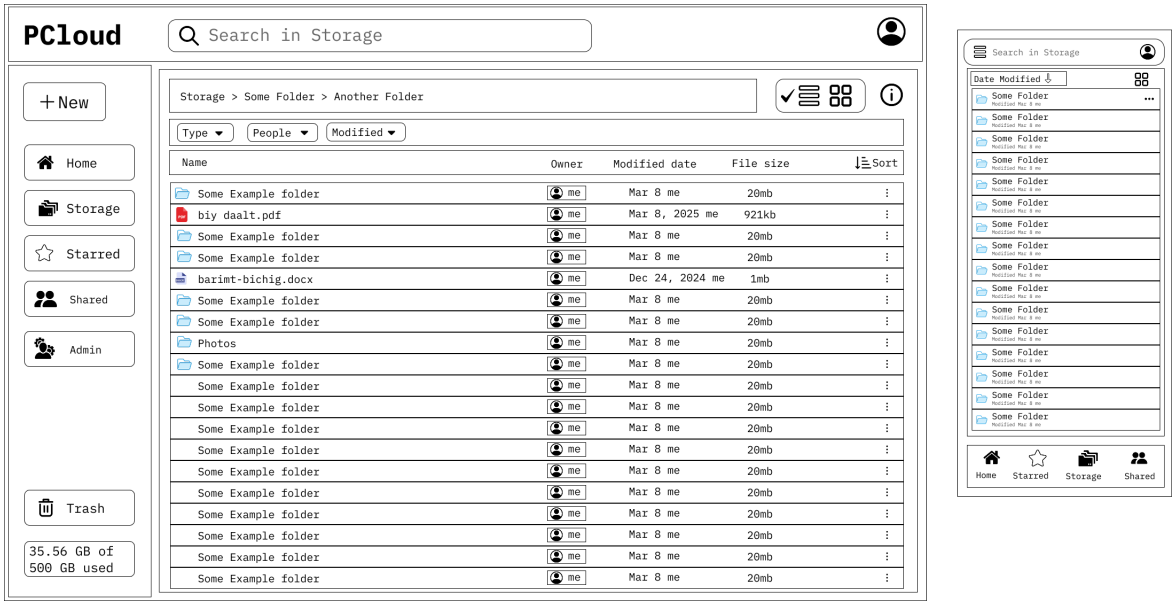
Зураг 3.4: Нэвтрэх дэлгэц

Энгийн хэрэглэгчийн нүүр хуудсан дээр файл болон хавтсын бүтэц харагдаж, upload/download, rename, delete, create folder зэрэг үндсэн үйлдлүүдийг хийх боломжтой. Мөн хайлт, харагдац өөрчлөх, shared файлууд харах зэрэг функцууд багтсан.



Зураг 3.5: Энгийн хэрэглэгчийн үндсэн хуудас

Админ хэрэглэгчийн хувьд дээрх боломжууд дээр нэмээд хэрэглэгч удирдлага, эрхийн бодлого тохируулах, системийн төлөв хянах цэсүүдийг ашиглах боломжтой.



Зураг 3.6: Админ хэрэглэгчийн нүүр хуудас

3.5 Хөгжүүлсэн системийн интерфейс

Хөгжүүлсэн интерфейс нь прототипийн үндсэн урсгалыг хадгалсан бөгөөд дараах дараалаар ажиллана.

1. Анхны тохиргоогоор админ хэрэглэгч болон хадгалалтын үндсэн хавтас үүсгэнэ (Зураг 3.3).
2. Хэрэглэгч серверийн хаяг, хэрэглэгчийн нэр, нууц үгээр нэвтэрнэ (Зураг 3.4).
3. Энгийн хэрэглэгч файлуудаа удирдаж, хуваалцсан агуулгад хандана (Зураг 3.5).
4. Админ хэрэглэгч хэрэглэгч болон системийн ерөнхий тохиргоог удирдана (Зураг 3.6).

3.6 Системд хийгдсэн тест

Системийн баталгаажуулалтыг дараах түвшинд гүйцэтгэсэн.

1. **Функциональ тест**: Нэвтрэх, файл байршуулах, татах, нэр өөрчлөх, устгах, эрх олгох урсгалуудыг usecase тус бүрээр шалгасан.
2. **Эрхийн тест**: Админ болон энгийн хэрэглэгчийн эрх ялгарах, зөвшөөрөлгүй хандалтыг блоклох шалгалт хийсэн.
3. **Интерфейсийн тест**: Веб болон мобайл клиент дээр үндсэн дэлгэцүүдийн хэрэглээний урсгал, алдааны мэдэгдлийг шалгасан.

3.7 Системийн нэвтрүүлэлт

Системийг дотоод туршилтын орчинд self-hosted хэлбэрээр нэвтрүүлж, серверийн локал storage directory-т өгөгдөл хадгалах байдлаар ажиллуулсан. Нэвтрүүлэлтийн үе шатанд хэрэглэгчийн бүртгэл үүсгэх, файл байршуулах, эрх хуваалцах, татах урсгалуудыг бодит хэрэглээний сценариар туршиж, ажиллагааны алдаануудыг засварласан.

3.8 Бүлгийн дүгнэлт

Энэ бүлэгт хувийн үүлэн хадгалалтын системийн архитектур, үндсэн компонентууд, өгөгдлийн зохиомж, прототип болон хэрэгжүүлсэн интерфейсийг тайлбарлалаа. Системийн шийдэл нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах self-hosted зарчимд тулгуурласан бөгөөд веб болон мобайл клиентээр дамжуулан нэгдсэн

API-р ажиллахаар зохион байгуулагдсан. Мөн функцийн болон эрхийн түвшний тестүүдээр системийн үндсэн урсгалуудыг баталгаажуулсан.

Ерөнхий дүгнэлт

Энэхүү дипломын ажлын хүрээнд хэрэглэгч өөрийн төхөөрөмж дээр байршуулж ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийн зохиомж, хөгжүүлэлт, туршилтыг хийж гүйцэтгэлээ. Төслийн гол зорилго нь хэрэглэгчийн өгөгдлийг гуравдагч талын серверт бус өөрийн хяналтын орчинд хадгалах, мөн веб болон мобайл клиентээр дамжуулан тухайн өгөгдөлд найдвартай хандах боломж бүрдүүлэхэд оршсон.

Судалгааны шатанд нийтийн болон self-hosted үүлэн хадгалалтын шийдлүүдийг харьцуулж, бага нөөцтэй орчинд хэрэгжих боломжтой архитектурын шаардлагыг тодорхойлсон. Шинжилгээний шатанд системийн функциональ шаардлага, хэрэглэгчийн үүрэг ба эрхийн загвар, гол usecase-уудыг боловсруулсан. Зохиомж, хөгжүүлэлтийн шатанд серверийн API, файл удирдлагын логик, эрхийн хяналт, нэвтрэлтийн урсгал, админ болон энгийн хэрэглэгчийн интерфейсийг хэрэгжүүлсэн.

Үр дүнд нь анхны тохиргоо, хэрэглэгч нэвтрэх, файл байршуулах/татах, хавтас удирдах, хуваалцах эрх олгох зэрэг үндсэн урсгалууд тогтвортой ажиллах хувийн үүлэн хадгалалтын системийг бий болгосон. Цаашид сүлжээний орчинд алсаас найдвартай холбогдох шийдлийг өргөжүүлэх, гүйцэтгэл болон аюулгүй байдлын тестийг илүү өргөн хүрээнд хийх боломжтой.

ХАВСРАЛТ А

Серверийн API-ийн жишээ

A.1 REST endpoint-ууд

Энэ хавсралтад хувийн үүлэн хадгалалтын системийн сервер талын үндсэн API endpoint-уудын жишээг үзүүлэв.

```
POST   /api/auth/login
POST   /api/auth/logout

GET    /api/users/me
POST   /api/users              // admin: create user
DELETE /api/users/{userId}     // admin: delete user

GET    /api/files?path=/       // list files/folders
POST   /api/files/upload       // upload file
GET    /api/files/download/{id} // download file
PATCH  /api/files/{id}/rename // rename file/folder
DELETE /api/files/{id}         // delete file/folder

POST   /api/shares             // grant permission
DELETE /api/shares/{shareId} // revoke permission
```

Ашигласан материалын жагсаалт

[1] Threadgold consulting. *Personal Cloud Storage Usage*. Personal Cloud Storage Usage report. 2025. URL: https://threadgoldconsulting.com/research/personal-cloud-storage-usage (**urlseen** 01/03/2026).

[2] Dean Hildebrand, Denis Serenyi. *Colossus under the hood: a peek into Google's scalable storage system*. Google's Scalable Storage Solution. 2021. URL: https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system (**urlseen** 28/03/2026).

[3] Google Workspace. *Google Workspace security whitepaper: Technology with security at its core*. Google Workspace Security Whitepaper. 2026. URL: https://workspace.google.com/learn-more/security/security-whitepaper/page-4/ (**urlseen** 29/03/2026).

[4] Google Workspace Admin Help. *Choose a geographic location for your data*. Google Workspace Data Region Policy Guide. 2026. URL: https://support.google.com/a/answer/14310028?hl=en (**urlseen** 29/03/2026).

[5] Google Workspace Admin Help. *Protect your organization's data with CSE*. Google Workspace Client-side Encryption Overview. 2026. URL: https://support.google.com/a/answer/14326936?hl=en (**urlseen** 29/03/2026).

[6] Google Developers. *Google Drive API: Track changes for users and shared drives*. Google Drive API Documentation. 2026. URL: https://developers.google.com/workspace/drive/api/guides/manage-changes (**urlseen** 29/03/2026).

[7] Google Developers. *Google Drive API: Receive notifications for resource changes*. Google Drive API Push Notification Guide. 2026. URL: https://developers.google.com/workspace/drive/api/guides/push (**urlseen** 29/03/2026).

[8] Google Developers. *Google Drive API: Manage uploads*. Google Drive API Upload Guide. 2026. URL: https://developers.google.com/drive/api/v3/manage-uploads (**urlseen** 29/03/2026).

[9] Google Developers. *Google Drive API: Manage file revisions*. Google Drive API Revision Management Guide. 2026. URL: https://developers.google.com/workspace/drive/api/guides/manage-revisions (**urlseen** 29/03/2026).

[10] Google Drive Help. *Check activity and file versions*. Google Drive Version History and Activity Guide. 2026. URL: https://support.google.com/drive/answer/2409045?hl=en (**urlseen** 29/03/2026).

[11] Nextcloud. *Nextcloud Files - Open source file sync and share platform*. Official Nextcloud Files overview page. 2026. URL: https://nextcloud.com/files/ (**urlseen** 29/03/2026).

[12] Nextcloud. *Core ownCloud Contributors Fork ownCloud Into Nextcloud*. Official Nextcloud launch announcement. 2016. URL: https://nextcloud.com/blog/press_releases/pr20160602/ (**urlseen** 29/03/2026).

[13] Nextcloud GmbH. *Nextcloud Architecture Whitepaper*. Whitepaper. Accessed: 2026-03-01. Nextcloud, **july** 2018. URL: https://nextcloud.com/c/uploads/2022/03/Architecture-Whitepaper-WebVersion-072018.pdf.

[14] Nextcloud. *Configuring External Storage (GUI)*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/external_storage_configuration_gui.html (**urlseen** 29/03/2026).

[15] Nextcloud. *Files access control*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/30/admin_manual/file_workflows/access_control.html (**urlseen** 29/03/2026).

[16] Nextcloud. *Server-side Encryption*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/encryption_configuration.html (**urlseen** 29/03/2026).

[17] Nextcloud. *Controlling file versions and aging*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/file_versioning.html (**urlseen** 29/03/2026).

[18] Nextcloud. *Deleted Items (trash bin)*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/trashbin_configuration.html (**urlseen** 29/03/2026).

[19] Nextcloud. *Transactional file locking*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/31/admin_manual/configuration_files/files_locking_transactional.html (**urlseen** 29/03/2026).

[20] Nextcloud. *Memory caching*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_server/caching_configuration.html (**urlseen** 29/03/2026).

[21] Nextcloud. *Nextcloud System Requirements*. NextCloud official Documentation. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/installation/system_requirements.html (**urlseen** 28/03/2026).

[22] ownCloud. *Scalable, secure, faster and more efficient: ownCloud Infinite Scale is here*. Official ownCloud Infinite Scale release announcement. 2022. URL: https://owncloud.com/news/infinitescalerelease/ (**urlseen** 29/03/2026).

[23] ownCloud. *Architecture and Concepts of Infinite Scale*. ownCloud Infinite Scale Architecture Documentation. 2026. URL: https://doc.owncloud.com/ocis/next/architecture/architecture.html (**urlseen** 29/03/2026).

[24] ownCloud. *Security Aspects of Infinite Scale*. ownCloud Infinite Scale Security Documentation. 2026. URL: https://doc.owncloud.com/ocis/next/security/security.html (**urlseen** 29/03/2026).

[25] ownCloud. *Availability and Scalability*. ownCloud Infinite Scale Availability and Scaling Guide. 2026. URL: https://doc.owncloud.com/ocis/next/availability_scaling/availability_scaling.html (**urlseen** 29/03/2026).

[26] Rust. *The Rust Programming Language*. Book about Rust. 2026. URL: https://doc.rust-lang.org/book/ (**urlseen** 02/03/2026).